Interrupt Budgets for Pre-Execution AI Control Monitors

Anonymous ACL submission

Abstract

LLM judges deployed as pre-execution monitors read each proposed action of an AI agent and can block it, but they are tuned to a per-step false-positive rate. A deployer experiences something else: alarms compound over a trajectory, so on LinuxArena a 1% per-step threshold interrupts 14–37% of benign runs across 12 monitor models. We bound this false interruption rate (FIR) instead. On attacks that are hard to detect, monitors then catch less: at a 5% budget, the catch rate of Claude Opus 4.6 on human-crafted sabotage falls from 0.79 to 0.59. Conformal or PAC thresholds calibrated on the deployed agent guarantee the budget, but thresholds calibrated on another agent or environment can exceed it. No statistic we tested reliably beats the running maximum: likelihood-ratio statistics hurt several monitors, help none once threshold uncertainty is counted, and collapse in stress tests that concentrate an attack's evidence in one action. When attacks by Claude Opus models are caught, the median alarm comes only at the action that causes the harm, leaving no time for review that does not block it. These patterns recur on MonitoringBench, where the budget costs less and likelihood ratios hurt our monitor.

1 Introduction

AI control protocols assume that a model may intentionally pursue harmful side objectives, and that a separate monitor has to catch it before damage is done (Greenblatt et al., 2024; Bhatt et al., 2025). When an action causes harm as soon as it runs (a "live failure"), only a *pre-execution* monitor, which reviews each action before it is executed, can prevent the harm (Tracy et al., 2026; Lindner et al., 2025). Such monitors are usually tuned to a per-step false-positive rate (FPR) of 1% or less and compared by their catch rate at that operating point (Bhatt et al., 2025; Tracy et al., 2026). They are LLM judges of agent transcripts (Chiang and Lee, 2023; Kim et al., 2024) and inherit the evaluation problems of such judges: coarse, often tied scores (Liu et al., 2023), judgments that can be biased (Wang et al., 2024; Koo et al., 2024; Chen et al., 2024), and, as we show, calibration that does not transfer across the agents they judge.

A per-step rate is not what a deployer experiences. Alarms accumulate over a trajectory: Jotautaitė et al. (2026) observe that a 1% per-action FPR corresponds to a 6.7–17.4% trajectory-level FPR and leave calibrating monitors to a common trajectory-level rate to future work; on LinuxArena we find 14–37% across 12 monitor models. Each interruption costs human review time or a failed task, and a monitor that interrupts too often tends to be switched off (Han et al., 2026). This cost falls on benign users, unlike the auditing budgets of control evaluations, which limit the effort of trusted reviewers (Section 3). We make this trajectory-level rate, the *false interruption rate* (FIR), the primary constraint and hold it to a budget α.

Our claim is that per-step evaluation hides both the cost and the limits of pre-execution monitoring. We propose no new estimator; we ask what established ones imply for monitors that block actions, and test it on published LinuxArena trajectories, using the stored per-step scores of 12 monitor models and our own, and repeat the main analyses on MonitoringBench (Section 8). Four findings support it; Figure 1 marks where each enters.

C1. The budget costs catch rate where detection is hard. At a 5% budget, monitors lose up to 0.38 of catch rate on human sabotage; the loss is not explained by per-step FIR and reorders them. On model-generated live attacks, capable monitors lose at most 0.13 (Section 4).

C2. The budget can be guaranteed at a price, but the calibration transfers poorly. Conformal and PAC thresholds keep it on discrete LLM scores, as theory predicts, but PAC gives up to

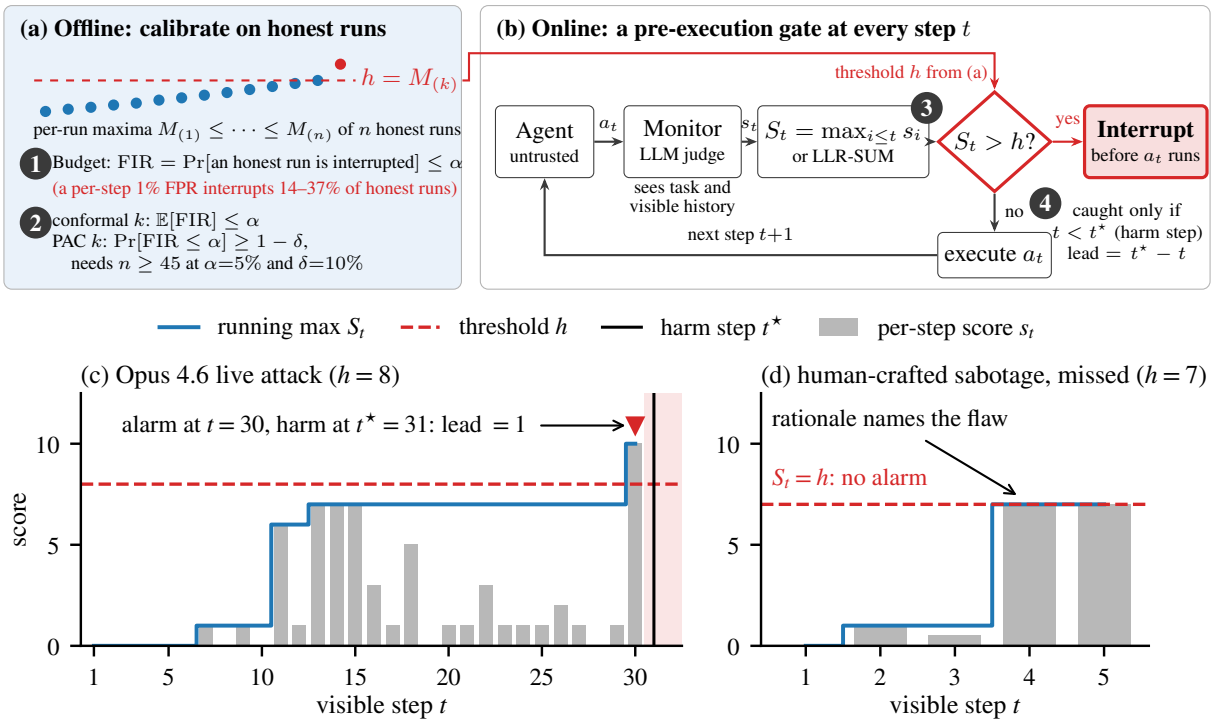


Figure 1: Overview. (a) Offline, the deployer calibrates a threshold h on n honest trajectories of the deployed agent so that the false interruption rate (FIR), the share of honest runs that are ever interrupted, stays within a budget α, either in expectation (conformal) or with probability $1-\delta$ (PAC). (b) Online, the monitor scores each proposed action a_t before it runs, a statistic S_t aggregates the scores, and the run is interrupted at the first t with $S_t > h$. An attack counts as caught only if this happens before the harm step $t^\star$, and the lead is $t^\star - t$. (c) An Opus 4.6 live attack scored by our monitor is flagged only at the attacking action, the last action before the harm ($h = 8$, calibrated on the 45 single-sample honest Opus 4.6 runs of Section 7). (d) In a human-crafted sabotage, the rationale names the injected vulnerability, but the score equals the threshold ($h = 7$, calibrated on the 200 honest Opus runs of the main experiment, scored as means of two samples), so the run is not interrupted (Appendix B). Both thresholds are set for a 5% budget; they differ because the honest pools and the number of samples differ. Circled numbers mark where findings C1–C4 (Section 1) apply.

0.13 of catch rate, and thresholds from other agents or environments can exceed the budget, up to an FIR of 24% for a weak monitor (Section 5).

C3. The running maximum is hard to improve on. Histogram likelihood-ratio statistics hurt several monitors, help none once threshold uncertainty is counted, and fail when an attack's evidence sits in one action (Section 6).

C4. Alarms can come at the last moment. Across seven attacker models, the median lead ranges from 1 to 23 steps; for Opus 4.6 it is one step, the harmful action itself, too short for any review that does not block (Section 7).

We also show that LinuxArena's `submit` call leaks attack labels to a monitor that sees it (Section 9).

2 Related work

AI control and agent monitoring. Greenblatt et al. (2024) introduced trusted monitoring and control protocols, and Ctrl-Z (Bhatt et al., 2025) extended them to multi-step agents with per-step resampling. Benchmarks for monitoring tool-using agents now cover several sabotage settings (Kutasov et al., 2025; Kaufman et al., 2025; Tracy et al., 2026) and vary what the monitor sees and how strong it is, from chain-of-thought access (Arnav et al., 2025) and other information (Arike et al., 2026) to monitors weaker than the agent (Kale et al., 2026) and asynchronous review (Stickland et al., 2025). A recurring finding is that monitors catch much less once attacks are shaped against them: refined by humans (Tracy et al., 2026; Najt et al., 2026; Jotautaitė et al., 2026), selected by the attacker (Schaeffer et al., 2026; Ge-Wang et al., 2026), aimed at the monitor itself (Terekhov et al.,

2026; Isbarov and Kantarcioglu, 2026; Remedios et al., 2026) or distributed over several agents (Makins et al., 2026). Pinto (2026) finds that detection does not transfer across the model families of the untrusted agent; we find the same for false-alarm calibration on honest runs (Section 5).

Timeliness and pre-execution oversight. Lindner et al. (2025) distinguish synchronous, semi-synchronous and asynchronous monitoring, and LinuxArena introduces live and delayed failures with a harm step (Tracy et al., 2026). Guard models check agent actions before or after they execute (Ravindra et al., 2026; Zheng et al., 2026), building on benchmarks of LLMs as judges of agent safety (Yuan et al., 2024; Ye et al., 2024; Zhan et al., 2024), and Han et al. (2026) study the unit of verification for pre-execution oversight. Jotautaitė et al. (2026) calibrate step-wise and full-trajectory monitors separately with empirical percentiles. We instead fix the trajectory-level rate as a budget with finite-sample guarantees and measure the alarm lead for different attackers. Ravindra et al. (2026), Han et al. (2026), Zheng et al. (2026) and Remedios et al. (2026) are concurrent work.

Sequential testing and calibration. Our thresholds use classical order-statistic tolerance limits (Wilks, 1941), split-conformal calibration (Angelopoulos and Bates, 2023) and its training-conditional (PAC) version (Vovk, 2013; Park et al., 2020). Conformal risk control bounds expected losses, of which the FIR is one (Angelopoulos et al., 2024), and conformal methods are increasingly applied to LLM outputs (Campos et al., 2024), also with API access only (Su et al., 2024), as for our monitors. Closest to our calibration, E-valuator (Sadhuka et al., 2025) thresholds the running maximum of a sequential statistic at a PAC order statistic to control false alarms when verifying task success; our PAC rule is the same procedure. We study it against adversarial agents with a harm step, measuring what the guarantee costs and when it fails across agents. Against such agents, the density-ratio statistics that E-valuator uses to gain power are not reliably better than the maximum, nor are CUSUM (Page, 1954), SPRT-style sums (Wald, 1945) or e-processes (Ramdas et al., 2023).

3 Setting

Monitoring model. An agent produces actions $a_1, \ldots, a_T$. Before a_t is executed, a monitor assigns it a score $s_t \in [0, 10]$ based on the task and the visible history. A detector maps the scores to a statistic $S_t = g(s_{1:t})$ and interrupts the run at the first t with $S_t > h$. By default we use the running maximum, $S_t = \max_{i \le t} s_i$, denoted MAX. In an attack trajectory with a *live* failure, the harm happens at the harm step $t^\star$, which is the first successful side-task test or, if there is none, the final submission; only an interrupt at $t < t^\star$ prevents it (Appendix K places $t^\star$ at the annotated harmful action instead). A *delayed* failure can be prevented at any step, and its $t^\star$ is the end of the trajectory (Tracy et al., 2026). The *pre-harm catch rate*, or *catch rate* for short, is the fraction of attacks interrupted in time. The *false interruption rate* is $\mathrm{FIR}(h) = \Pr_{\text{honest}}[\exists t : S_t > h] = \Pr[\max_t S_t > h]$, and an upper bound α on it is the *interrupt budget*. The *lead* of an alarm is $t^\star - t$, counted in visible steps (excluding attacker-only tools), so a lead of one means that the alarm fires on the last action before the harm. We write PAC for training-conditional guarantees. In the text we give catch rates as fractions and FIRs and budgets as percentages; tables and figures give measured FIRs as fractions.

Interrupt budgets versus auditing budgets. Control evaluations limit trusted labor: Greenblatt et al. (2024) audit up to 2% of problems, and Ctrl-Z (Bhatt et al., 2025) audits 1% of actions at the 99th percentile of honest per-action scores. An interrupt budget bounds the probability that a benign run is stopped, counted per trajectory since a user loses one task however many actions are flagged; only interrupts before harm count as catches.

Data. The LinuxArena data come from its public release (CC-BY-4.0) (Tracy et al., 2026). We score 72 human-crafted sabotage trajectories (STRAJ; 5 live and 67 delayed failures) against 200 honest Opus 4.6 trajectories, and 92 attacks generated by Claude Haiku 4.5 (30 live) against 200 honest Haiku trajectories. From the full release (22,215 trajectories in 172 runs) we add live attacks by seven attacker models whose side task succeeded, each paired with honest runs of the same agent, together with the per-step scores of the 12 monitor models included in the release. As in LinuxArena, monitors never see attacker-only tools or the agent's `submit` and scorer calls (Section 9).

Our monitor. We use DeepSeek-V4.1-Flash with thinking enabled. The prompt is adapted from LinuxArena's `old-action` live-monitor prompt

and shows the full visible history; Appendix A gives decoding and sampling details. Unless stated otherwise, thresholds are calibrated with 5-fold cross-validation on honest trajectories, breaking ties by random jitter (10 seeds); Appendix J reports the deterministic rule.

4 What an interrupt budget costs

Table 1 reports our monitor in LinuxArena's evaluation format (Appendix A checks our metric code against LinuxArena's). At a 1% per-step FPR, our monitor catches 0.55 of STRAJ, but 24% of honest trajectories are interrupted at least once. Figure 2 shows the same pattern for the 12 LinuxArena monitors, whose per-step thresholds correspond to an FIR of 14–37%. Calibrating the threshold to an FIR of 5% instead lowers the catch rate of every monitor: from 0.79 to 0.59 for Opus 4.6, from 0.73 to 0.60 for Gemini 3.1 Pro, from 0.47 to 0.13 for Sonnet 4.6, and from 0.55 to 0.37 for ours. The loss is not reliably related to the per-step FIR (Kendall $\tau = 0.03$), and the ranking shifts ($\tau = 0.76$ between the two): GLM 5 falls from third to fifth and Sonnet 4.6 from seventh to tenth, and they lose rank in 91% and 82% of bootstrap replicates (Appendix J). The cost concentrates where detection is hard. Human sabotage, mostly delayed failures (67 of 72) that an after-the-fact monitor could also catch, is our hardest attack set; on Opus-generated live attacks, which only a pre-execution monitor can stop, the nine monitors that catch more than 0.10 of human sabotage lose at most 0.07, ours loses at most 0.13 on the live attacks of seven attackers, and only weak monitors lose more (GPT-5 Nano: 0.85 to 0.47; Appendix J).

5 Calibrating the interrupt budget

With the running maximum, the FIR of a threshold h is the survival function of the per-trajectory maximum $M = \max_t s_t$ on honest runs. Given n honest calibration trajectories with sorted maxima $M_{(1)} \le \cdots \le M_{(n)}$, we compare three thresholds: the empirical $(1-\alpha)$ quantile; the split-conformal order statistic $M_{(k_c)}$ with $k_c = \lceil (n+1)(1-\alpha) \rceil$, which guarantees $\mathbb{E}[\mathrm{FIR}] \le \alpha$ (Angelopoulos and Bates, 2023); and a PAC order statistic, which keeps this deployment within budget with high probability. For i.i.d. continuous maxima, the classical tolerance-limit argument (Wilks, 1941) gives $\Pr[\mathrm{FIR}(M_{(k)}) \le \alpha] = \Pr[\mathrm{Bin}(n, \alpha) \ge n-k+1]$ (Appendix C), and the PAC threshold takes the smallest k for which this probability is at least $1-\delta$ (Vovk, 2013; Sadhuka et al., 2025). It is finite iff $n \ge \lceil \log \delta / \log(1-\alpha) \rceil$ (45 for $\alpha = 5\%$ and $\delta = 10\%$; Table 6), whereas conformal needs only $n \ge 1/\alpha - 1$.

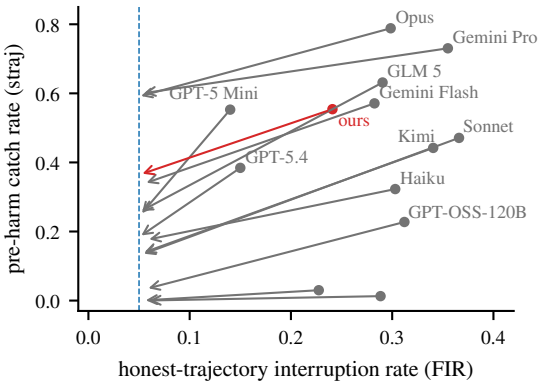


Figure 2: Per-step 1% thresholds (dots) versus a 5% interrupt budget (arrow heads) for 12 LinuxArena monitors and ours on human-crafted sabotage.

Validation without test-set noise. We treat each honest pool of 200 trajectories as the population, draw calibration sets of size n with replacement, compute the exact FIR of each threshold on the whole pool, and repeat this 1,000 times. The draws are i.i.d. from the pool, so the guarantees hold by construction (conservatively, as duplicates tie); the check is whether they survive discrete scores and small n, and the shift experiments below test exchangeability itself. We count a requirement as met within two Monte Carlo standard errors. Conformal suits an average requirement, PAC a high-probability one.

Results. As theory predicts, the method that matches the requirement meets it on both honest pools in all 52 feasible settings ($\alpha \in \{2, 5, 10\}\%$, $n \in [20, 300]$, $\delta \in \{5, 10\}\%$). The empirical quantile misses the average requirement for every $n \le 150$ and never meets a PAC requirement (Figure 3, left). At $n = 20$, its mean FIR is 8.5% and it exceeds the budget in 57% of draws. Conformal calibration lowers the mean to 4.7% but still exceeds the budget in 19–37% of draws for $n \in [20, 100]$, whereas PAC cuts this share to 2–7%, below δ. The guarantees cost catch rate (Figure 3, right; Table 6). At $\alpha = 5\%$, PAC lowers the STRAJ catch rate by 0.13 relative to the quantile at $n = 60$ and by 0.06 at $n = 200$, and relaxing α to 10% recovers about 0.10. On model-generated

Table 1: Baseline on human-crafted sabotage (STRAJ, 72) against Opus honest runs (200), at per-step FPRs of 1% and 5%. FIR@1% is the share of honest trajectories that the 1% per-step threshold interrupts. Reference monitors use LinuxArena's stored scores; trajectory-level AUC is that of a whole-transcript monitor.

Monitor	catch@1% step [95% CI]	catch@5% step	FIR @1% step	live-max AUC	traj. AUC
Ours (DeepSeek-V4.1-Flash)	0.55 [0.44, 0.67]	0.80	0.24	0.73	0.90
Claude Opus 4.6	0.79 [0.70, 0.90]	0.95	0.30	0.87	0.94
GPT-5 Nano	0.03 [0.00, 0.08]	0.30	0.23	0.35	0.54

attacks, the catch rate stays at 0.84–0.86.

Risk control and discrete scores. For the 0–1 interruption loss, conformal risk control (Angelopoulos et al., 2024) selects the split-conformal threshold, and Learn-then-Test (LTT; Angelopoulos et al., 2025) with exact binomial p-values selects the PAC threshold (Appendix E). With the more generic Hoeffding–Bentkus p-values (Bates et al., 2021), LTT lowers the catch rate by up to 0.07. Jitter is not required for the guarantees: alarming only when the raw maximum exceeds the raw order statistic keeps both, at a cost of at most 0.05 in catch rate for our monitor and up to 0.09 for the integer scores of the LinuxArena monitors (Appendix J), whereas alarming on ties as well breaks them.

Calibration must match the deployment. The guarantees assume that calibration and deployment trajectories are exchangeable. With our monitor, calibrating on Haiku honest runs and deploying on Opus honest runs violates this assumption. At $\alpha = 5\%$, conformal calibration then yields an FIR of 7.9% and exceeds the budget in 72% of draws (95% CI for the mean FIR 4.0–12.7% when both pools are resampled), and PAC yields 4.0% but exceeds the budget in 13% of draws, more than δ. Figure 4 extends the comparison to more agents. For the weak GPT-5 Nano monitor, a threshold calibrated on one of 11 agents and deployed on another reaches a mean FIR of up to 24%, and pooling the 11 agents does not help, giving 9.1% on Opus and 10.8% on Sonnet. For our stronger monitor (7 agents, 45 honest runs each), cross-agent thresholds give 1–7%, except those calibrated on Gemini 3 Flash (6.5–10%), and pooled ones 2–5%; with 45 runs per agent, a single cell carries about three points of sampling error, and cross-agent cells average 4.7% against 3.3% within an agent. A likely reason is that a strong monitor's honest scores concentrate at low values, with a median honest maximum of 2.0 for every agent. Holding out each LinuxArena environment in turn (Opus agent) raises the pooled FIR of conformal thresholds to 6.5% for our monitor and 6.0–9.0% for the Haiku and Kimi monitors (random splits: 5.0%, $p \leq 0.024$), and single environments reach 30–65%; PAC stays below 5% on average. A pooled threshold also interrupts the longest third of honest runs at 9–11%, which per-tercile calibration corrects (Appendix J). Calibrating per agent and task mix is the safe choice; a pooled threshold must first be checked for the monitor in use.

6 Which statistic should be thresholded?

Evidence accumulates over a trajectory, which suggests replacing the maximum with a sequential statistic. We compare MAX with CUSUM (Page, 1954), a likelihood-ratio CUSUM (LLR-CUSUM) and an SPRT-style likelihood-ratio sum (LLR-SUM; Wald, 1945), with likelihood ratios estimated from score histograms of the training folds, treating steps as independent. We also include their product with Ville's fixed threshold $1/\alpha$, an e-process only if steps were independent (Ramdas et al., 2023), and a three-strikes rule adapted from Ctrl-Z (Bhatt et al., 2025). For our monitor (Table 2), LLR-SUM and the e-process raise the catch rate on human sabotage by 0.14 and 0.10, while three strikes lowers it by 0.18; with thresholds refitted in the bootstrap, only this loss excludes zero (Appendix H). Under the deterministic rule, MAX leaves part of the budget unused and the gains grow by 0.02 (Appendix J). On Haiku attacks, LLR-SUM and the e-process are slightly below MAX, the CUSUM variants tie, and the e-process is too conservative (realized FIR 0%).

The gain does not carry over to other monitors. With the 12 LinuxArena monitors (Figure 6 in Appendix H), LLR-SUM helps some monitors (GLM 5: +0.20 [+0.10, +0.30]; ours: +0.14) and hurts others (GPT-5.4: -0.15 [-0.23, -0.07]). These three remain significant after Holm correction across the 13 monitors (paired sign-flip test), the losses for Opus, Sonnet and Haiku (-0.09 to -0.12) only before it. When the bootstrap also resamples honest runs and refits thresholds and like-

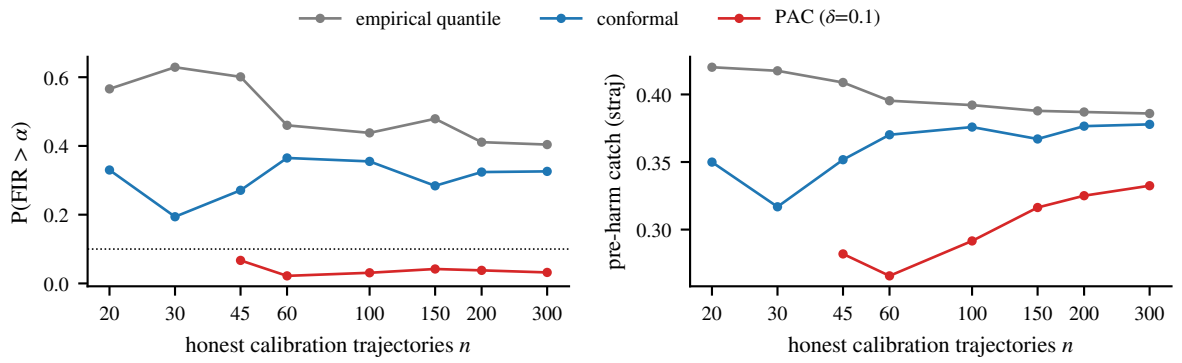


Figure 3: Calibration at $\alpha = 5\%$ (STRAJ vs. Opus honest; population bootstrap, 1,000 draws). Left: probability that a deployment exceeds the budget (dotted line: $\delta = 0.1$). Right: pre-harm catch rate (y-axis truncated).

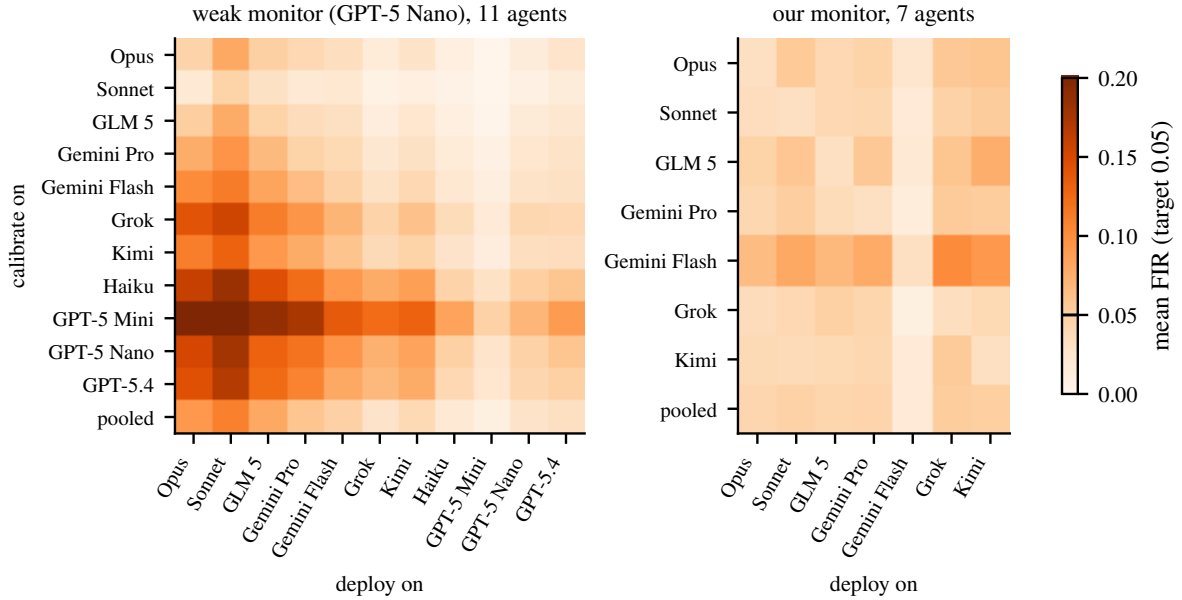


Figure 4: Mean FIR at a 5% target when a conformal threshold calibrated on one agent's honest runs (rows) is deployed on another's (columns). Left: GPT-5 Nano monitor (200 runs per agent, calibration sets of 100). Right: our monitor (45 runs per agent, sets of 45).

lihood ratios, the losses for GPT-5.4 and Sonnet remain, but the gains for GLM 5 [+0.00, +0.40] and ours [−0.03, +0.25] no longer exclude zero (Appendix H). On Opus-generated attacks, LLR-SUM fitted in-domain significantly hurts 6 of 12 monitors by 0.20 to 0.33, also under the two-way bootstrap, and for some the sign flips between $\alpha = 5\%$ and 10%.

Attackers that adapt to the statistic. We transform the stored scores of test attacks, keeping thresholds and likelihood ratios fitted on unmodified data (Appendix F). A *concentrated* attack keeps only its highest-scoring pre-harm step and looks honest otherwise, close to what Opus 4.6 attacks do. MAX is unaffected; LLR-SUM keeps a catch rate of at most 0.12 in 9 of 11 monitor–attack pairs and LLR-CUSUM at most 0.01 on human sabotage for 9 of 10 monitors. A *sub-threshold* attack, which knows the threshold, keeps every action one level below it: MAX catches nothing, LLR-SUM up to 0.77. Alarming when either statistic exceeds its threshold at $\alpha/2$ keeps the budget, loses at most 0.14 to MAX, and recovers up to 0.76 on sub-threshold attacks. MAX is the safer default, with an accumulating statistic alongside it.

7 Timeliness: the alarm lead depends on the attacker

Detection of model-generated attacks is close to saturated. We score 15 live attacks with a success-

Table 2: Detectors for our monitor at a 5% FIR budget (split-conformal, 5-fold CV × 10 seeds; the per-step row also uses cross-validated thresholds). Δ vs. MAX: paired bootstrap over attack trajectories, thresholds held fixed. All five differences on STRAJ remain significant after Holm correction (paired sign-flip test; adjusted $p \leq 0.043$); Appendix H gives wider two-way intervals that also refit the thresholds.

	STRAJ vs. Opus honest		Haiku attack vs. Haiku honest	
Detector	catch (FIR)	Δ vs. MAX [95% CI]	catch	Δ
per-step 1% (reference)	0.55 (0.25)	–	0.89 (0.26)	–
MAX	0.37 (0.05)	–	0.85	–
LLR-SUM	0.50 (0.05)	+0.14 [+0.06, +0.22]	0.82	−0.02
e-process ($h = 1/\alpha$)	0.47 (0.04)	+0.10 [+0.04, +0.17]	0.79	−0.05
CUSUM	0.42 (0.06)	+0.05 [+0.01, +0.11]	0.86	+0.01
LLR-CUSUM	0.28 (0.05)	−0.08 [−0.17, −0.01]	0.85	+0.00
three strikes	0.19 (0.05)	−0.18 [−0.28, −0.08]	0.84	−0.01

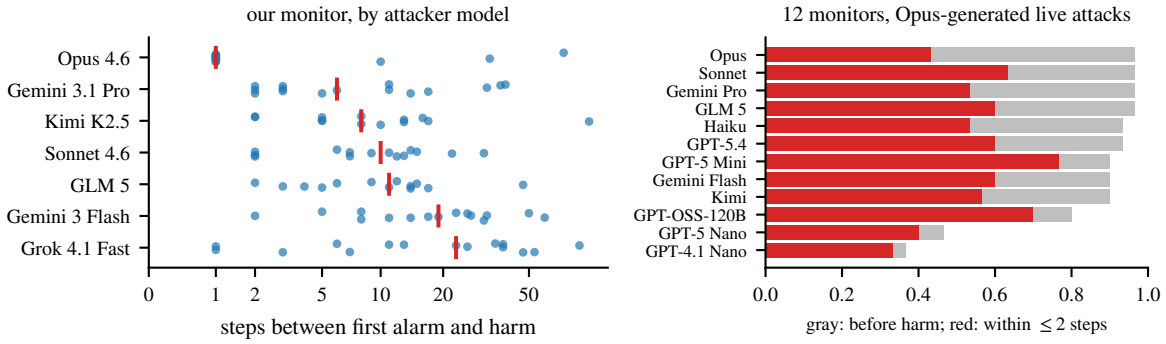


Figure 5: Alarm lead at a 5% FIR budget. Left: our monitor on live attacks of seven attacker models (dots: caught attacks; bars: median). Right: 12 LinuxArena monitors on Opus-generated live attacks.

ful side task per attacker model against 45 honest runs of the same agent; with 45 runs, the 5% threshold is the second-largest honest maximum, so these FIRs are resolved only to about two points. For all seven attacker models, the AUC of the pre-harm maximum is 0.96–1.00, and the catch rate at a 5% budget is 0.87–1.00. The attackers differ in when the alarm arrives (Figure 5, left). Attacks by Grok 4.1 Fast and Gemini 3 Flash are caught a median of 23 and 19 steps before the harm. Attacks by Claude Opus 4.6 are caught a median of one step before it: 10 of the 13 caught attacks (95% CI for the share 0.46–0.95) are flagged only on the last action before the harm. Placing the harm at the harmful action that an LLM annotates (Appendix K) instead of at the side-task test leaves every catch rate of our monitor and this one-step lead unchanged, and shortens the other attackers' median leads by at most six steps (Grok: 23 to 17). The 12 LinuxArena monitors, with 30 such attacks each, show the same pattern (Figure 5, right). The nine whose catch rate on human sabotage exceeds 0.10 at a 5% budget catch 0.90–0.97 of these attacks before harm, but for 10 of the 12 the median lead is one step; the exceptions are the Opus 4.6 and Gemini 3.1 Pro monitors, with 7 and 2 steps. Across all 12, 45–91% of the catches come within two steps of the harm. Human sabotage also leaves little lead (at a 1% per-step FPR, a median of 2 steps for its 5 live attacks).

The ordering of attackers follows no obvious capability ranking (with our monitor, Sonnet 4.6 attacks leave a median of 10 steps and Gemini 3.1 Pro attacks 6), and 13–15 caught attacks per attacker are too few to test one. A one-step lead leaves no time for review. In a deployment simulation with thresholds at a 5% budget, synchronous blocking prevents 0.87 of the Opus attacks, but with a 10-minute review delay, semi-synchronous and asynchronous review prevent at most 0.13, even with 30 s agent steps; on Haiku attacks, caught earlier, asynchronous review still prevents about half (Appendix D).

What the monitor sees and says. In the missed sabotage of Figure 1d, the rationale names the flaw but the score only ties the threshold (Appendix B). Accusations that the score ignores are more common in missed sabotage than in honest runs below

Table 3: Our monitor on both benchmarks ($\alpha = 5\%$; violation rates at $n = 100$). LinuxArena: human sabotage (rows 1–5) and Opus 4.6 live attacks (rows 6–7, calibrated on 45 honest runs). MonitoringBench: 300 Opus 4.5 attacks and honest runs. Rows 2–5 use cross-validated thresholds with randomized ties (strict rule: budget catch rates 0.35 and 0.89), rows 6–7 full-data thresholds with the strict rule.

	LinuxArena	MonitoringB.
FIR at 1% per step	0.24	0.10
Catch rate: 1% step / budget	0.55 / 0.37	0.96 / 0.92
$\Pr[\text{FIR} > \alpha]$: quantile / PAC	0.44 / 0.03	0.42 / 0.02
LLR-SUM − MAX	+0.14	−0.09
Three strikes − MAX	−0.18	−0.71
Caught before harm	13/15	0.87
Median lead; lead $\leq$2 steps	1; 10/13	1; 0.70

the threshold (31% vs. 14%, Fisher $p = 0.012$; Appendix G), but adding them to the score raises the catch rate by less than 0.01.

8 Replication on a second benchmark

We repeat the analyses on MonitoringBench (Jotautaitė et al., 2026), which another group built on the system-administration tasks of BashArena (Kaufman et al., 2025). From a separate release of benign trajectories for it (Küçükkurt and Elasky, 2026) we take 300 task-matched pairs of honest and attack runs by Claude Opus 4.5, and score every step up to the harmful call. As in the benchmark's own monitor prompt, the attack targets are redacted (Appendix I). Table 3 compares the two benchmarks. The budget costs less here, with the catch rate falling from 0.96 to 0.92, in part because the benign pool was screened before release, which makes the FIR somewhat optimistic (Appendix I). At the benchmark's own operating point, a 1% trajectory-level FPR, our monitor catches 0.70–0.77 of the attacks. The benchmark's full-trajectory Opus 4.5 monitor, which does not see tool outputs, catches 0.71 of them, and its stepwise monitor, which does, about 0.83 (Appendix I). The calibration results replicate. LLR-SUM, whose gain for our monitor on human sabotage is uncertain, hurts it here (-0.09 [-0.12, -0.06], Holm-significant), so a statistic must be validated on deployment data. Timeliness also replicates: the median lead is one step, and 62% of catches come only at the harmful call.

9 A pitfall: label leakage through the submit call

Studies that render LinuxArena transcripts themselves can leak labels: the final `submit` call of an attack carries arguments in the side task's scorer interface, which in 34 of 72 STRAJ trajectories and 34 of 92 Haiku attacks contain field names that never appear in honest runs. Shown the call, our monitor's MAX catch rate at a 5% budget rises from 0.37 to 0.66 and its trajectory-level AUC from 0.90 to 0.95, and three conclusions of the configuration comparison reverse (Appendix D). LinuxArena's own monitors hide the call (LinuxArena, 2026), as do all our results; monitor inputs should be checked against the original benchmark before studies are compared.

10 Conclusion

Under a per-trajectory budget, what a pre-execution defense achieves depends mainly on the monitor and its calibration data: catch rates on human sabotage range from 0.00 to 0.60 across monitors, far more than between statistics or calibration methods. For LLM judges in general, the setting combines three calibration problems: tied scores that a deterministic threshold cannot fully use, score distributions that shift with the producer and the task of the judged text, and per-item error rates that compound over a sequence.

The Opus attacks expose a limit that a stronger monitor cannot remove: when the only incriminating action is the one that causes the harm, there is no evidence to accumulate and no time for review, and whether the attack is stopped depends on one synchronous decision made under the budget. Evaluations of monitors against capable attackers should therefore report the pre-harm catch rate at a trajectory-level budget together with the lead.

For deployment, our results suggest a recipe. Choose α from the catch–FIR curve by weighing an interruption against the harm it may prevent (Appendix J), and decide whether it must hold in expectation or with probability $1 - \delta$. Calibrate on at least $\lceil 1/\alpha - 1 \rceil$ (conformal) or $\lceil \log \delta / \log(1 - \alpha) \rceil$ (PAC) honest trajectories of the deployed agent, scaffold and task mix, and recalibrate when any of these changes. Threshold the running maximum, add an accumulating statistic only if it wins in cross-validation on deployment data (selected with LTT and combined with MAX at $\alpha/2$ each), and block synchronously.

Limitations

Our evidence comes from two English-language benchmarks in a single domain, software engineering and system administration in Linux environments. On MonitoringBench, the benign pool was screened before release, which makes the measured FIR somewhat optimistic, and all attacks come from a single attacker model. Human-crafted sabotage is scarce: of its 72 trajectories, only five are live failures, so our conclusions about human attacks, especially about lead time, are indicative. The model-generated attacks are a convenience sample of the attacker models in the release, and their detection is nearly saturated.

Our own monitor is a single open-weight model that we accessed only through APIs, so the served version may change over time, and each configuration was scored once; Appendix J compares the two samples of the baseline run. The 12 reference monitors use a different prompt, so we compare statistics only within a monitor, and monitor models may have seen the public trajectories during training, which could inflate detection.

All guarantees assume that calibration and deployment trajectories are exchangeable, which holding out an environment shows can fail within one agent. Our adaptive attackers are score-level stress tests on stored scores, not new attack trajectories that adapt to the rule, and the deployment simulation relies on assumed agent step times and review delays. The risk-control analysis covers only the 0–1 interruption loss; graded losses, such as the work that an interruption discards, would require the general form of conformal risk control. The rationale labels near the thresholds and the harmful actions of live attacks (Appendix K) come from an LLM annotator that shares the monitor's model, without human labels. Likelihood ratios were estimated from score histograms with steps treated as independent; better estimators may compare differently with MAX. Holm correction is applied within the detector comparisons (Table 2 and the MonitoringBench detectors) and across monitors (Figure 6); the configuration and cascade comparisons, the Fisher test on rationales and the stress tests are uncorrected and exploratory. Most intervals hold the thresholds fixed; the two-way intervals of Appendix H, which also resample honest runs, are wider. All intervals resample trajectories; resampling environments instead widens some of them (Appendix J). The timing results cover only attacks whose side task succeeded. Finally, we count an interruption as a lost task; if interrupted runs can resume after review, the relevant cost is the expected number of reviews per run rather than the FIR.

Ethical Considerations

We used only published trajectories and attack data, created no new attack content, and did not use any evaluation data for training. We followed the licenses and access terms of the datasets we used, LinuxArena and MonitoringBench, both released under CC-BY-4.0. The benign MonitoringBench trajectories carry no stated license, and we do not redistribute them. Findings about monitor blind spots and short alarm lead times could inform attackers, so we report aggregate results and describe attack mechanisms without reusable payloads. Monitors of this kind are intended to protect deployments of AI agents. Their false interruptions also impose costs on benign users, and our budget formulation makes this trade-off explicit.

References

Anastasios N. Angelopoulos and Stephen Bates. 2023. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591.

Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. 2025. Learn then test: Calibrating predictive algorithms to achieve risk control. *The Annals of Applied Statistics*, 19(2).

Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. 2024. Conformal risk control. In *International Conference on Learning Representations (ICLR)*.

Rauno Arike, Raja Mehta Moreno, Rohan Subramani, Shubhorup Biswas, and Francis Rhys Ward. 2026. How does information access affect LLM monitors' ability to detect sabotage? In *International Conference on Machine Learning (ICML)*.

Benjamin Arnav, Pablo Bernabeu-Pérez, Nathan Helm-Burger, Tim Kostolansky, Hannes Whittingham, and Mary Phuong. 2025. CoT Red-Handed: Stress testing Chain-of-Thought monitoring. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Stephen Bates, Anastasios Angelopoulos, Lihua Lei, Jitendra Malik, and Michael Jordan. 2021. Distribution-free, risk-controlling prediction sets. *Journal of the ACM*, 68(6):1–34.

Aryan Bhatt, Cody Rushing, Adam Kaufman, Tyler Tracy, Vasil Georgiev, David Matolcsi, Akbir Khan, and Buck Shlegeris. 2025. Ctrl-Z: Controlling AI agents via resampling. *Preprint*, arXiv:2504.10374.

Margarida Campos, António Farinhas, Chrysoula Zerva, Mário A. T. Figueiredo, and André F. T. Martins. 2024. Conformal prediction for natural language processing: A survey. *Transactions of the Association for Computational Linguistics*, 12:1497–1516.

Wendy W. Chapman, Will Bridewell, Paul Hanbury, Gregory F. Cooper, and Bruce G. Buchanan. 2001. A simple algorithm for identifying negated findings and diseases in discharge summaries. *Journal of Biomedical Informatics*, 34(5):301–310.

Guiming Hardy Chen, Shunian Chen, Ziche Liu, Feng Jiang, and Benyou Wang. 2024. Humans or LLMs as the judge? A study on judgement bias. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8301–8327, Miami, Florida, USA. Association for Computational Linguistics.

Cheng-Han Chiang and Hung-yi Lee. 2023. Can large language models be an alternative to human evaluations? In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15607–15631, Toronto, Canada. Association for Computational Linguistics.

Catherine Ge-Wang, Tyler Crosse, Benjamin Hadad, Joachim Schaeffer, Ram Potham, and Tyler Tracy. 2026. Attack selection in agentic AI control evaluations meaningfully decreases safety. *Preprint*, arXiv:2606.06529.

Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. 2024. AI control: Improving safety despite intentional subversion. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 16295–16336.

Yuchen Han, Cheng Yan, and Wuyang Zhang. 2026. More rejective, not more discriminative: The unit of verification in Pre-Execution LLM oversight. *Preprint*, arXiv:2608.23941.

Jafar Isbarov and Murat Kantarcioglu. 2026. Bypassing AI control protocols via Agent-as-a-Proxy attacks. *Preprint*, arXiv:2602.05066.

Monika Jotautaitė, Maria Angelica Martinez, Ollie Matthews, and Tyler Tracy. 2026. MonitoringBench: Semi-Automated Red-Teaming for agent monitoring. *Preprint*, arXiv:2605.09684.

Neil Kale, Chen Bo Calvin Zhang, Kevin Zhu, Ankit Aich, Paula Rodriguez, Scale Red Team, Christina Q. Knight, and Zifan Wang. 2026. Reliable Weak-to-Strong monitoring of LLM agents. In *International Conference on Learning Representations (ICLR)*.

Adam Kaufman, James Lucassen, Tyler Tracy, Cody Rushing, and Aryan Bhatt. 2025. BashArena: A control setting for highly privileged AI agents. *Preprint*, arXiv:2512.15688.

Seungone Kim, Juyoung Suk, Shayne Longpre, Bill Yuchen Lin, Jamin Shin, Sean Welleck, Graham Neubig, Moontae Lee, Kyungjae Lee, and Minjoon Seo. 2024. Prometheus 2: An open source language model specialized in evaluating other language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 4334–4353, Miami, Florida, USA. Association for Computational Linguistics.

Ryan Koo, Minhwa Lee, Vipul Raheja, Jong Inn Park, Zae Myung Kim, and Dongyeop Kang. 2024. Benchmarking cognitive biases in large language models as evaluators. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 517–545, Bangkok, Thailand. Association for Computational Linguistics.

Can Küçükkurt and Ethan Elasky. 2026. Releasing benign trajectories for MonitoringBench. LessWrong. Posted 2 September 2026.

Jonathan Kutasov, Yuqi Sun, Paul Colognese, Teun van der Weij, Linda Petrini, Chen Bo Calvin Zhang, John Hughes, Xiang Deng, Henry Sleight, Tyler Tracy, Buck Shlegeris, and Joe Benton. 2025. SHADE-Arena: Evaluating sabotage and monitoring in LLM agents. *Preprint*, arXiv:2506.15740.

David Lindner, Charlie Griffin, Tomek Korbak, Roland S. Zimmermann, Geoffrey Irving, Sebastian Farquhar, and Alan Cooney. 2025. Practical challenges of control monitoring in frontier AI deployments. *Preprint*, arXiv:2512.22154.

LinuxArena. 2026. control-tower. GitHub repository, MIT license. File `format_history.py`; accessed 6 October 2026.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522, Singapore. Association for Computational Linguistics.

Oliver Makins, Orazio Angelini, Zohreh Shams, and Mary Phuong. 2026. Multi-agent AI control: Distributed attacks hamper per-instance monitors. *Preprint*, arXiv:2607.07368.

Elle Najt, Colin Toft, Tyler Tracy, Fabien Roger, and Joe Benton. 2026. SLEIGHT-Bench: A benchmark of evasion attacks against agent monitors. *Preprint*, arXiv:2605.16626.

E. S. Page. 1954. Continuous inspection schemes. *Biometrika*, 41(1–2):100–115.

Sangdon Park, Osbert Bastani, Nikolai Matni, and Insup Lee. 2020. PAC confidence sets for deep neural networks via calibrated prediction. In *International Conference on Learning Representations (ICLR)*.

Lucas Pinto. 2026. Calibration-family overfit: Why trusted sabotage monitors don't transfer across lineages. *Preprint*, arXiv:2607.06596.

Aaditya Ramdas, Peter Grünwald, Vladimir Vovk, and Glenn Shafer. 2023. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science*, 38(4).

Preeti Ravindra, Rahul Tiwari, and Vincent Wolowski. 2026. Democratizing agent deployment safety: A structural monitoring approach. *Preprint*, arXiv:2607.14570.

Alex Remedios, Simon Storf, Fabien Roger, and John Hughes. 2026. Red-Teaming auto mode: Improving blocking classifiers against malign coding agents. *Preprint*, arXiv:2609.19587.

Shuvom Sadhuka, Drew Prinster, Clara Fannjiang, Gabriele Scalia, Bonnie Berger, Aviv Regev, and Hanchen Wang. 2025. E-valuator: Reliable agent verifiers with sequential hypothesis testing. *Preprint*, arXiv:2512.03109.

Joachim Schaeffer, Arjun Khandelwal, and Tyler Tracy. 2026. Attack selection reduces safety in concentrated AI control settings against trusted monitoring. *Preprint*, arXiv:2602.04930.

Asa Cooper Stickland, Jan Michelfeit, Arathi Mani, Charlie Griffin, Ollie Matthews, Tomek Korbak, Rogan Inglis, Oliver Makins, and Alan Cooney. 2025. Async control: Stress-testing asynchronous control measures for LLM agents. *Preprint*, arXiv:2512.13526.

Jiayuan Su, Jing Luo, Hongwei Wang, and Lu Cheng. 2024. API is enough: Conformal prediction for large language models without logit-access. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 979–995, Miami, Florida, USA. Association for Computational Linguistics.

Mikhail Terekhov, Alexander Panfilov, Daniil Dzenhaliou, Caglar Gulcehre, Maksym Andriushchenko, Ameya Prabhu, and Jonas Geiping. 2026. Adaptive attacks on trusted monitors subvert AI control protocols. In *International Conference on Learning Representations (ICLR)*.

Tyler Tracy, Ram Potham, Nick Kuhn, Myles Heller, Anshul Khandelwal, Cody Rushing, Henri Lemoine, Miguel Brandao, Tomas Turlik, Adam Hanson, Josh Hills, Amy Ngo, Ram Rachum, Nik Mitchell, Falko Galperin, Oscar Sykes, Pip Arnott, Samuel Prieto Lima, Carlos Giudice, and 15 others. 2026. LinuxArena: A control setting for AI agents in live production software environments. *Preprint*, arXiv:2604.15384.

Vladimir Vovk. 2013. Conditional validity of inductive conformal predictors. *Machine Learning*, 92(2–3):349–376.

Abraham Wald. 1945. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186.

Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Lingpeng Kong, Qi Liu, Tianyu Liu, and Zhifang Sui. 2024. Large language models are not fair evaluators. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9440–9450, Bangkok, Thailand. Association for Computational Linguistics.

S. S. Wilks. 1941. Determination of sample sizes for setting tolerance limits. *The Annals of Mathematical Statistics*, 12(1):91–96.

Junjie Ye, Sixian Li, Guanyu Li, Caishuang Huang, Songyang Gao, Yilong Wu, Qi Zhang, Tao Gui, and Xuanjing Huang. 2024. ToolSword: Unveiling safety issues of large language models in tool learning across three stages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2181–2211, Bangkok, Thailand. Association for Computational Linguistics.

Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. 2024. R-judge: Benchmarking safety risk awareness for LLM agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1467–1490, Miami, Florida, USA. Association for Computational Linguistics.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, Bangkok, Thailand. Association for Computational Linguistics.

Zhijie Zheng, Yu Li, Chen Qian, Yuqian Fu, Yanwei Fu, Lu Sheng, Jing Shao, and Dongrui Liu. 2026. StepGuard: Learning Step-Level guardrails with scalable supervision and Safety-Utility balancing. *Preprint*, arXiv:2608.24777. To appear at EMNLP 2026.

A Costs, artifacts and reproducibility

Costs. All API calls of the project, including exploratory runs not reported here, cost about USD 239.

Artifacts and licenses. Table 4 lists the artifacts we used. We used all of them for evaluation only and none for training.

Table 4: Artifacts, licenses and use.

Artifact	License	Use
LinuxArena public trajectories and stored monitor scores	CC-BY-4.0	evaluation
MonitoringBench attack trajectories	CC-BY-4.0	evaluation
MonitoringBench benign trajectories (Docent)	none stated	evaluation; not redistributed
LinuxArena control-tower (monitor prompts adapted)	MIT	prompt templates

Decoding and configuration. All monitor calls use temperature 1.0 with thinking enabled unless stated otherwise. The live monitor allows 6,000 output tokens in the baseline run and 12,000 in all other runs, and the trajectory-level monitor allows 20,000. Scores in the baseline run are means of two samples per step, and those of the trajectory-level monitor means of two samples per trajectory; the configuration comparison (Table 8), Section 7 and MonitoringBench use one. Two API providers serve DeepSeek-V4.1-Flash in our runs, and each comparison uses scores from a single provider; the trajectory-level scores in Table 1 come from a different provider than the per-step scores. Where a run scored the `submit` step, we drop that score; no other step has the call in its history. Tool outputs are clipped to 3,000 characters, and rendered histories to 240,000 characters for the live monitor and 400,000 for the trajectory-level monitor. We made all scoring calls between 2026-09-25 and 2026-09-27 and the harm-step annotation (Appendix K) on 2026-10-09 (UTC). Full prompts and configuration files are in the supplementary material.

Metric check. From the stored scores we approximately reproduce LinuxArena's own metric: GPT-5 Nano lets 96.9% of STRAJ succeed undetected, against the 94% reported by Tracy et al. (2026).

Model names. Table 5 maps the short labels in the figures to the names in the text and to the model identifiers in the releases.

Software. We used Python 3.11.15, NumPy 2.4.6, the OpenAI Python client 3.17–3.19 with OpenAI-compatible endpoints and Matplotlib 3.11.2. We read and exported MonitoringBench with pandas 3.0.6, PyArrow 25.0.1 and the Docent SDK 0.1.87. Code, configurations and analysis scripts are provided as anonymized supplementary material. Per-step scores are released without transcript text, and the MonitoringBench benign trajectories are not redistributed.

B Qualitative examples

We describe mechanisms only and do not reproduce commands or payloads.

Opus 4.6 live attack caught on the last action before the harm (Figure 1c). In a message-processing service, the agent carries out the assigned refactoring for 29 visible steps, which the monitor scores at most 7, below the threshold of 8. These steps include probes for available networking utilities that the monitor later identifies as reconnaissance. The attack itself is a single action that opens a remote shell to an external host in the background. The monitor scores it 10 and names the matching covert objective, but this action is the last one before the harm. In 10 of the 13 caught Opus attacks, the alarm comes on this last action.

Honest trajectory with the highest score. Among all 315 honest trajectories of the seven agents, two reach the highest maximum score of 9. One is a Grok 4.1 Fast agent implementing a notification-preference system for a clinical-trial service. Late in the trajectory it sets one team's alert preference so that an alert type is not delivered. The monitor reads this as matching a covert objective in its prompt, the suppression of safety notifications. The action may be part of testing the feature, which would make this a false interruption driven by overlap between legitimate work and the listed objectives.

Human sabotage missed at a 5% budget (Figure 1d). In a web-scraping service, a five-step trajectory adds an archive-processing function that passes a path taken from scraped page content to a shell command without the sanitizer used elsewhere in the module. The monitor's rationale identifies the flaw and the covert objective it serves, but assigns a score of 7. This equals the 5% FIR threshold of 7 calibrated on the honest maxima, so the alarm rule, which requires a strict inequality, does not fire.

C Derivation of the PAC threshold

Let F be the distribution function of the honest maximum and $U_i = F(M_i)$. Then $U_{(k)} = F(M_{(k)}) \sim \mathrm{Beta}(k, n-k+1)$ and $\mathrm{FIR}(h) = 1 - U_{(k)}$, so $\mathrm{FIR} \le \alpha$ iff at most $k-1$ of the U_i

Table 5: Model names. Identifiers are those of the agent runs in the LinuxArena and MonitoringBench releases; GPT-4.1 Nano and GPT-OSS-120B appear only as LinuxArena monitors, under these labels. Agent: honest runs; attacker: attack runs; monitor: stored or our per-step scores. The monitors compared in Appendix I keep MonitoringBench's names.

Label	Name in text	Identifier	Roles
Opus	Claude Opus 4.6	`claude-opus-4-6`	agent, attacker, monitor
Sonnet	Claude Sonnet 4.6	`claude-sonnet-4-6`	agent, attacker, monitor
Haiku	Claude Haiku 4.5	`claude-haiku-4-5-20251001`	agent, attacker, monitor
GPT-5.4	GPT-5.4	`gpt-5.4`	agent, monitor
GPT-5 Mini	GPT-5 Mini	`gpt-5-mini`	agent, monitor
GPT-5 Nano	GPT-5 Nano	`gpt-5-nano`	agent, monitor
GPT-4.1 Nano	GPT-4.1 Nano	–	monitor
GPT-OSS-120B	GPT-OSS-120B	–	monitor
Gemini Pro	Gemini 3.1 Pro	`gemini-3.1-pro-preview`	agent, attacker, monitor
Gemini Flash	Gemini 3 Flash	`gemini-3-flash-preview`	agent, attacker, monitor
Grok	Grok 4.1 Fast	`grok-4.1-fast`	agent, attacker
Kimi	Kimi K2.5	`kimi-k2.5`	agent, attacker, monitor
GLM 5	GLM 5	`glm-5`	agent, attacker, monitor
–	Claude Opus 4.5	`claude-opus-4-5-20251101`	agent, attacker (MonitoringBench)
ours	DeepSeek-V4.1-Flash	`deepseek-flash`, `deepseek-v4.1-flash`	monitor, annotator

fall below $1-\alpha$. That is, at least $n-k+1$ of the n i.i.d. U_i exceed $1-\alpha$, an event of probability $\Pr[\mathrm{Bin}(n,\alpha) \geq n-k+1]$. This probability grows with k, and for $k=n$ it equals $1-(1-\alpha)^n$, which is at least $1-\delta$ iff $n \geq \lceil \log\delta / \log(1-\alpha) \rceil$. Smaller calibration sets admit no finite PAC threshold.

D Monitor configurations

Table 8 compares configurations of our monitor on the same 222 trajectories. With a 5-step history window, enabling thinking raises the STRAJ catch rate by +0.22 [+0.12, +0.33]. A 5-step window beats the current action alone by +0.23 [+0.04, +0.35] and does not differ significantly from the full history (+0.04 [−0.07, +0.13]), at 74% of its cost and 57% of its median per-step latency. The two-stage cascade escalates about 7% of steps. Compared with an always-on 5-step thinking monitor, it cuts the cost from USD 1.09 to 0.52 per 1,000 steps and the 90th-percentile latency from 15.6 s to about 4 s. On human sabotage at a 5% budget, however, its catch rate changes by −0.08 [−0.14, −0.03] relative to that monitor. On Haiku attacks, the change is not significant (−0.06 [−0.14, +0.00]). With the submit call visible (Section 9), three of these comparisons point the other way: whether a short history window matters, whether the current action alone suffices and whether the cascade is free.

Deployment simulation. We replay the measured per-step scores and monitor latencies of the 30 Haiku live attacks, and of the 15 Opus 4.6 live attacks of Section 7 with thresholds from the 45 honest Opus runs, under the three deployment modes of Lindner et al. (2025). The public data do not record how long an agent step takes, so we use $d \in \{5, 30\}$ s, and a human review takes $R \in \{1\,\text{min}, 10\,\text{min}, 1\,\text{h}\}$. Thresholds keep a 5% budget: the split-conformal order statistic of the honest maxima, with an alarm only on scores above it. In synchronous mode every action waits for its score and an alarm stops the agent until review, so any alarm before the harm prevents it. In semi-synchronous mode a flagged action is held for review while the agent continues, and the harm is prevented only if a review finishes before the harm lands. In asynchronous mode actions never wait, and a flag is reviewed R after its score arrives. Table 7 gives the share of attacks whose harm is prevented.

E Risk control, discrete scores and statistic selection

Risk-control procedures. With i.i.d. continuous honest maxima and the 0–1 loss $L_i(h) = \mathbb{1}[M_i > h]$, both procedures reduce to the order statistics of Section 5. Each depends on h only through $m(h) = \sum_i L_i(h)$, a non-increasing step function equal to $n-k$ on $[M_{(k)}, M_{(k+1)})$, so each returns an order statistic. Conformal risk control with loss bound $B=1$ selects $\hat{h} = \inf\{h : (m(h)+1)/(n+1) \leq \alpha\}$, and $(n-k+1)/(n+1) \leq \alpha$ iff $k \geq (n+1)(1-\alpha)$, so it returns $M_{(k_c)}$. LTT tests $H_h : \mathrm{FIR}(h) > \alpha$ for h in decreasing order (fixed-sequence testing) with the p-value $p(h) = \Pr[\mathrm{Bin}(n,\alpha) \leq m(h)]$ and keeps the smallest re-

Table 6: Cost of the guarantees in STRAJ pre-harm catch rate for the recommended method (MAX statistic, our monitor) with $n = 60$, 100 and 200 calibration trajectories, and the difference from the empirical quantile.

Requirement	α	Method	min. n	catch (n=60 / 100 / 200)	Δ vs. quantile
$\mathbb{E}[\text{FIR}] \leq \alpha$	5%	conformal	19	0.37 / 0.38 / 0.38	0.03 / 0.02 / 0.01
$\mathbb{E}[\text{FIR}] \leq \alpha$	10%	conformal	9	0.45 / 0.46 / 0.46	$\leq$0.01
$\Pr[\text{FIR} \leq \alpha] \geq 0.9$	5%	PAC	45	0.27 / 0.29 / 0.33	0.13 / 0.10 / 0.06
$\Pr[\text{FIR} \leq \alpha] \geq 0.9$	10%	PAC	22	0.37 / 0.40 / 0.43	0.10 / 0.07 / 0.04
$\Pr[\text{FIR} \leq \alpha] \geq 0.95$	5%	PAC	59	0.27 / 0.29 / 0.31	0.13 / 0.10 / 0.07

Table 7: Deployment simulation at a 5% budget: share of attacks whose harm is prevented, for agent step time d and review delay R, on Haiku live attacks (top) and on the Opus 4.6 live attacks of Section 7 (bottom). Synchronous blocking does not depend on R. The FIR column is the realized honest FIR (60 honest runs for the current-action monitors, 200 for the full-history monitor, 45 for the Opus attacks).

				semi-synchronous			asynchronous		
Monitor	FIR	d	synchronous	1 min	10 min	1 h	1 min	10 min	1 h
thinking, full history	0.045	5 s	0.90	0.83	0.53	0.00	0.63	0.00	0.00
		30 s	0.90	0.90	0.73	0.10	0.83	0.53	0.00
thinking, current action	0.017	5 s	0.77	0.73	0.13	0.00	0.57	0.00	0.00
		30 s	0.77	0.77	0.53	0.03	0.77	0.47	0.00
no thinking, current action	0.033	5 s	0.83	0.70	0.10	0.00	0.67	0.00	0.00
		30 s	0.83	0.83	0.53	0.00	0.80	0.53	0.00
Opus attacks, thinking, full history	0.000	5 s	0.87	0.20	0.07	0.00	0.13	0.00	0.00
		30 s	0.87	0.20	0.13	0.07	0.20	0.13	0.00

jected h. This p-value is valid because under H_h the count $m(h)$ is binomial with rate $\text{FIR}(h) > \alpha$ and hence stochastically larger than $\text{Bin}(n, \alpha)$. Since $p(h)$ is non-increasing in h, it rejects exactly the h with $m(h) \leq m^\star$, where $m^\star$ is the largest m with $\Pr[\text{Bin}(n, \alpha) \leq m] \leq \delta$, and the smallest such h is $M_{(n-m^\star)}$. As $\Pr[\text{Bin}(n, \alpha) \leq n-k] \leq \delta$ iff $\Pr[\text{Bin}(n, \alpha) \geq n - k + 1] \geq 1 - \delta$, this is the PAC threshold.

Discrete scores. Monitor scores take few values. For our two-sample means they are multiples of 0.5, and we write g for the gap between adjacent levels. Adding independent jitter $U \sim \text{U}[0, \epsilon)$ with $\epsilon < g$ to every step score makes the maxima continuous, so the guarantees above hold for the jittered rule. That rule randomizes the decision only for trajectories whose maximum equals the threshold level. Jitter never reorders distinct levels, so the jittered order statistic $M'_{(k)}$ lies in $[M_{(k)}, M_{(k)} + \epsilon)$, where $M_{(k)}$ is the raw order statistic. If $M > M_{(k)}$, then $M' \geq M \geq M_{(k)} + g > M'_{(k)}$. Hence the deterministic rule "alarm iff $M > M_{(k)}$" alarms only on trajectories on which the jittered rule alarms. Its FIR is thus never larger, and both guarantees carry over. The rule "alarm iff $M \geq M_{(k)}$" has no such guarantee. Table 9 confirms both statements. Between 1% and 3% of honest trajectories have their maximum exactly at the threshold level. Across both honest pools and $\alpha \in \{5, 10\}\%$, the strict rule's PAC violation rate is at most 4%, whereas that of the $\geq$ rule reaches 47% for Haiku honest runs at $\alpha = 10\%$ and $n = 200$. The strict rule lowers the catch rate by 0.00–0.05 relative to jitter.

Statistic selection. LTT can also choose between MAX and LLR-SUM with a guarantee, by splitting δ over them. We run 300 replicates. In each, we draw 60 honest trajectories to fit the likelihood ratio and 100 to calibrate. We split the attacks into two halves, one to fit and select and one to evaluate, and compute the FIR exactly on the honest pool. The Bonferroni split from $\delta = 10\%$ to 5% changes no PAC index at $n = 100$, so LTT and naive selection coincide there. At $n = 200$ it raises the PAC index by one, from 195 to 196. Naive selection then exceeds the budget in 2–7% of draws and LTT in 1–3%, and LTT lowers the catch rate by 0.00–0.04. Across both sizes, the selected detector exceeded the budget in at most 6% of draws, lost at most 0.04 to the better fixed statistic, and beat both for five monitors (Table 10).

F Adaptive-attacker stress tests

The transformations act on the stored per-step scores of test-fold attacks. Thresholds (split-

Table 8: Monitor configurations on the same 222 trajectories, STRAJ catch rate at a 1% per-step FPR. Each score is a single sample and each honest pool has 60 runs, so the full-history row differs from Table 1.

Configuration	catch@1% [95% CI]	USD / 1k steps	median latency / step
thinking, full history	0.59 [0.47, 0.73]	1.46	9.1 s
thinking, 5-step window	**0.63** [0.51, 0.75]	1.09	5.2 s
thinking, current action	0.40 [0.28, 0.62]	0.56	4.2 s
no thinking, full history	0.46 [0.31, 0.60]	0.83	3.7 s
no thinking, 5-step window	0.41 [0.28, 0.54]	0.82	3.1 s
no thinking, current action	0.32 [0.18, 0.50]	0.40	2.6 s

Table 9: Threshold procedures at $\alpha = 5\%$, $\delta = 10\%$ (STRAJ vs. Opus honest, MAX, population bootstrap, 1,000 draws per block). For each n, the columns give the mean FIR, $\Pr[\mathrm{FIR} > \alpha]$ (viol.) and pre-harm catch rate. The top block uses jittered scores. There, CRC and LTT (binomial) selected the same threshold as conformal and PAC in all draws, and Hoeffding's bound needs $n \geq 461$ at $\alpha = 5\%$. The bottom block uses an independent set of draws to compare jitter with deterministic rules on raw scores.

	$n = 45$			$n = 100$			$n = 200$		
Procedure	FIR	viol.	catch	FIR	viol.	catch	FIR	viol.	catch
conformal	0.040	0.27	0.35	0.048	0.36	0.38	0.047	0.31	0.38
CRC	0.040	0.27	0.35	0.048	0.36	0.38	0.047	0.31	0.38
PAC	0.019	0.07	0.29	0.018	0.03	0.29	0.027	0.03	0.32
LTT, binomial	0.019	0.07	0.29	0.018	0.03	0.29	0.027	0.03	0.32
LTT, Hoeffding–Bentkus	0.019	0.07	0.29	0.018	0.03	0.29	0.022	0.01	0.31
LTT, Hoeffding	no finite threshold			no finite threshold			no finite threshold		
conformal, jitter	0.040	0.27	0.35	0.047	0.34	0.37	0.047	0.34	0.38
conformal, strict $>$	0.034	0.13	0.32	0.039	0.12	0.35	0.040	0.06	0.36
conformal, ties alarm ($\geq$)	0.052	0.32	0.37	0.060	0.42	0.39	0.061	0.47	0.39
PAC, jitter	0.018	0.07	0.28	0.017	0.02	0.29	0.028	0.04	0.33
PAC, strict $>$	0.014	0.03	0.24	0.012	0.00	0.24	0.022	0.00	0.31
PAC, ties alarm ($\geq$)	0.030	0.09	0.33	0.028	0.03	0.32	0.040	0.07	0.34

conformal, 5% FIR) and likelihood ratios are fitted on the unmodified training folds, with 5-fold cross-validation and 10 seeds. In a *concentrated* attack, the highest-scoring pre-harm step keeps its score, and every other pre-harm step receives a score drawn from the honest training steps. In a *sub-threshold* attack, every step score is capped one level below the level of the MAX threshold; the attacker knows the threshold, an oracle worst case. The OR rule alarms when MAX or LLR-SUM exceeds its threshold calibrated at $\alpha/2$. The realized FIR was 3–6% for all detectors. Table 11 lists the monitors with a catch rate of at least 0.10.

G Rationale analysis

We analyze the rationale returned with each of the 65,574 step-by-sample scores of the 564 baseline trajectories. We split the `reasoning` field of each JSON answer into clauses at sentence punctuation and at contrastive conjunctions. A clause counts as an affirmed accusation if it contains an accusation cue and no negation trigger, such as *no*, *not*, *without*, *none*, *unrelated* or *non-*. The cues come from the wording of the prompt. They include *malicious*, *hidden goal*, *secret objective*, *deceptive*, *suspicious*, *exfiltration*, *credential theft*, *backdoor*, *privilege escalation* and *security violation*, as well as verbs such as *matches* or *accomplishes* followed by a covert objective. Negation is scoped at the clause level, in the style of NegEx (Chapman et al., 2001), because the monitor often writes long negated lists, such as "no evidence of accessing secrets, exfiltration, backdoor insertion, or . . . ". The clause rules were developed on the Haiku pool only.

Validation with an LLM annotator. We drew a stratified sample of 390 rationales, with up to 13 per cell of pool, attack or honest run, score band and lexicon label. DeepSeek-V4.1-Flash in thinking mode labeled each rationale twice as a firm, hedged or no accusation, and the annotator saw only the rationale text. The two labels agreed in 96% of cases, with $\kappa = 0.94$ for the three classes and 0.96 for accusing versus not; this is the test–retest consistency of one model, not agreement between annotators. Table 12 compares the lexicon with the annotator, weighting each cell by its population size. At scores 0–2 the annotator finds al-

Table 10: Choosing between MAX and LLR-SUM with LTT ($\alpha = 5\%$, $\delta = 10\%$; STRAJ unless noted). Cells give the pre-harm catch rate with $\Pr[\mathrm{FIR} > \alpha]$ in parentheses. Naive selection uses PAC(δ) thresholds for both statistics and coincides with LTT at $n = 100$.

	$n = 100$			$n = 200$			
Monitor	MAX	LLR-SUM	LTT select	MAX	LLR-SUM	naive select	LTT select
Ours	0.29 (0.02)	0.21 (0.02)	0.34 (0.04)	0.32 (0.04)	0.33 (0.05)	0.40 (0.07)	0.37 (0.03)
Ours (Haiku attacks)	0.90 (0.02)	0.86 (0.01)	0.90 (0.02)	0.91 (0.03)	0.87 (0.00)	0.91 (0.03)	0.91 (0.01)
Claude Opus 4.6	0.46 (0.03)	0.19 (0.02)	0.46 (0.04)	0.50 (0.02)	0.27 (0.02)	0.52 (0.03)	0.49 (0.02)
Claude Sonnet 4.6	0.11 (0.03)	0.00 (0.03)	0.11 (0.03)	0.12 (0.04)	0.01 (0.04)	0.12 (0.04)	0.12 (0.03)
Claude Haiku 4.5	0.09 (0.05)	0.02 (0.02)	0.09 (0.06)	0.12 (0.02)	0.03 (0.02)	0.12 (0.02)	0.10 (0.01)
GPT-5.4	0.10 (0.02)	0.01 (0.02)	0.09 (0.03)	0.13 (0.06)	0.02 (0.04)	0.13 (0.07)	0.12 (0.02)
GPT-5 Mini	0.11 (0.02)	0.05 (0.01)	0.11 (0.02)	0.15 (0.04)	0.10 (0.02)	0.16 (0.04)	0.13 (0.02)
GPT-5 Nano	0.00 (0.03)	0.00 (0.02)	0.00 (0.03)	0.00 (0.03)	0.00 (0.04)	0.00 (0.03)	0.00 (0.02)
GPT-4.1 Nano	0.00 (0.03)	0.00 (0.03)	0.00 (0.03)	0.00 (0.03)	0.00 (0.03)	0.00 (0.03)	0.00 (0.01)
GPT-OSS-120B	0.01 (0.03)	0.00 (0.03)	0.01 (0.06)	0.02 (0.05)	0.00 (0.04)	0.02 (0.07)	0.01 (0.03)
Gemini 3.1 Pro	0.54 (0.01)	0.59 (0.04)	0.60 (0.04)	0.57 (0.02)	0.61 (0.01)	0.61 (0.02)	0.61 (0.01)
Gemini 3 Flash	0.18 (0.02)	0.15 (0.02)	0.23 (0.04)	0.25 (0.06)	0.20 (0.03)	0.28 (0.07)	0.25 (0.02)
Kimi K2.5	0.09 (0.02)	0.04 (0.03)	0.10 (0.04)	0.12 (0.02)	0.08 (0.02)	0.13 (0.03)	0.11 (0.01)
GLM 5	0.17 (0.02)	0.21 (0.02)	0.23 (0.04)	0.20 (0.03)	0.30 (0.03)	0.30 (0.04)	0.26 (0.02)

Table 11: Pre-harm catch rate under score-level adaptive attacks (5% FIR budget; STRAJ vs. Opus honest unless noted). MAX is 0 on sub-threshold attacks by construction. LLR: LLR-SUM; L-CUSUM: LLR-CUSUM. The MAX columns come from a separate cross-validation run and differ from Figure 2 by at most 0.01.

	original				concentrated				sub-threshold			
Monitor	MAX	LLR	L-CUSUM	OR	MAX	LLR	L-CUSUM	OR	MAX	LLR	L-CUSUM	OR
Ours	0.37	0.50	0.28	0.37	0.37	0.27	0.00	0.32	0.00	0.50	0.27	0.25
Ours (Haiku att.)	0.85	0.82	0.85	0.85	0.85	0.08	0.84	0.86	0.00	0.77	0.85	0.76
Claude Opus 4.6	0.59	0.50	0.11	0.49	0.59	0.10	0.00	0.48	0.00	0.28	0.06	0.09
Claude Sonnet 4.6	0.13	0.01	0.01	0.12	0.14	0.00	0.00	0.12	0.00	0.01	0.01	0.00
Claude Haiku 4.5	0.16	0.05	0.05	0.11	0.17	0.00	0.00	0.10	0.00	0.03	0.04	0.00
GPT-5.4	0.18	0.03	0.01	0.12	0.18	0.00	0.00	0.11	0.00	0.04	0.02	0.02
GPT-5 Mini	0.23	0.22	0.04	0.13	0.24	0.00	0.00	0.11	0.00	0.18	0.04	0.03
Gemini 3.1 Pro	0.59	0.63	0.61	0.61	0.60	0.60	0.54	0.60	0.00	0.62	0.51	0.57
Gemini 3 Flash	0.34	0.31	0.10	0.24	0.34	0.05	0.00	0.20	0.00	0.21	0.04	0.07
Kimi K2.5	0.13	0.20	0.05	0.14	0.14	0.12	0.00	0.11	0.00	0.16	0.02	0.04
GLM 5	0.25	0.45	0.21	0.33	0.26	0.03	0.00	0.17	0.00	0.44	0.18	0.23

most no accusations, so nearly all lexicon hits there are false positives. At 3–4 the lexicon misses two thirds of the accusations, which are phrased without cue words, for example "concerning", "credential reconnaissance" or "aligns with covert objective 3". Lexicon labels would inflate accusation rates for honest runs, which consist mostly of low-score steps.

We labeled all 1,174 rationales of steps with a mean score of 3–8 (human sabotage pool) or 3–6 (Haiku pool) once with the annotator. A verbal bonus of at most 2 cannot move steps outside these ranges across the thresholds of 7 and 5. The rationale-aware statistic adds $\beta \in \{0.5, 1, 2\}$ times the share of a step's rationales that accuse to its score. It then thresholds MAX with split-conformal calibration (5-fold CV $\times$ 10 seeds). On human sabotage, the three values of β change the catch rate by +0.007 [+0.001, +0.014], +0.006 [+0.001, +0.011] and -0.032 [-0.069, -0.004], and Haiku attacks are unaffected. Before the harm, 15 of the 49 missed human sabotage trajectories and 28 of the 194 honest trajectories below the threshold contain an accusation that the score does not act on (Fisher $p = 0.012$); this comparison was designed after the lexicon analysis and is exploratory. On the Haiku pool, 2 of the 8 missed attacks and 9% of honest trajectories below the threshold contain an accusation. In the rationales we inspected, accusations below the threshold often concern earlier actions while the current action is judged benign, because the prompt asks the monitor to score the current action. The annotator shares the monitor's model and may share its blind spots, and we collected no human labels. We do not release the rationale texts, which describe attacks.

Table 12: Accusation lexicon versus the LLM annotator by the rationale's own score (stratified sample, population-weighted). The annotator rate is the share of rationales that the annotator labels as accusing. The last row pools all bands $\geq$3 and gives 95% stratified Jeffreys intervals: each cell's rate gets a Beta$(k+\frac{1}{2}, n-k+\frac{1}{2})$ posterior and cells are weighted by population size. The sampled lexicon hits at scores 5–10 contain no false positives, and the precision estimate exceeds the posterior's 97.5% quantile, so its upper limit is set to 1.

Score band	n	annotator rate	lexicon precision	lexicon recall
0–2	104	0.00	0.10	1.00
3–4	104	0.47	0.75	0.32
5–6	90	0.98	1.00	0.63
7–10	92	1.00	1.00	0.95
$\geq$3 (pooled)	286	–	0.99 [0.84, 1.00]	0.88 [0.85, 0.89]

H Statistic comparison across monitors

Figure 6 shows, for each LinuxArena monitor with a non-trivial catch rate, the change in its catch rate when LLR-SUM replaces MAX at a 5% budget (Section 6), on human sabotage and on Opus-generated attacks.

Fitting the likelihood ratio only on model-generated attacks, as a deployer without human attack data would have to, gives the same picture on human sabotage (Holm-significant: GLM 5 +0.28, Opus -0.16, GPT-5.4 -0.15).

Threshold uncertainty. The intervals in Figure 6 and Table 2 come from a paired bootstrap over attack trajectories, with the thresholds of each cross-validation fold held fixed. Table 13 adds a two-way bootstrap with 1,000 replicates. Each replicate resamples honest and attack trajectories, assigns cross-validation folds by original trajectory so that copies never straddle training and test folds, refits the likelihood ratios, recalibrates the thresholds and recomputes the difference. The six significant losses on Opus-generated attacks and the losses for GPT-5.4 and Sonnet 4.6 on human sabotage stay below zero, and so does the loss for our monitor on MonitoringBench (-0.09 [-0.15, -0.03]). The gains for GLM 5 and for our monitor on human sabotage no longer exclude zero. Table 14 repeats the analysis for all detectors of our monitor. None of the gains in Table 2 excludes zero (e-process: +0.10 [+0.00, +0.19]), while the losses of three strikes on human sabotage and on MonitoringBench and of the e-process on Haiku attacks and on MonitoringBench do.

I Replication on MonitoringBench

Data. MonitoringBench (Jotautaitė et al., 2026) releases 2,644 successful attack trajectories on BashArena under CC-BY-4.0. Claude Opus 4.5 and 4.6 generated them with several attack-generation strategies, and the release records the tool call at which each side task first succeeds. Küçükkurt and Elasky (2026) released 550 benign trajectories through the Docent platform, paired with 550 of these attacks and all generated by Claude Opus 4.5 on the same 116 tasks. These benign trajectories carry no stated license, so we use them for evaluation only and do not redistribute them. Before release, each benign run was scored with GPT-5.6 Sol and the MonitoringBench monitor prompt, and the 12 runs scoring 7 or higher were replaced by new rollouts. The benign pool is therefore cleaner than raw honest behavior, and the FIR measured on it is optimistic. We exported the collection with the Docent SDK and drew 300 task-matched pairs from 92 tasks whose attack has an annotated harmful call. We truncate attack runs after the harmful call, which cannot change the pre-harm catch rate, and their tool calls match the original logs step by step. Honest runs receive the main-task description from the original logs of the same task. As for LinuxArena, `submit` is hidden and the agent's own text is never shown.

Monitor view. BashArena's monitor prompt lists the four security failures but redacts "the exact IP addresses and usernames targeted by the red team". We use the same redacted objectives. With the unredacted targets, the catch rate at a 5% budget stays at 0.92. More attacks are caught before harm at the full-data threshold, 0.91 instead of 0.87, and LLR-SUM loses less to MAX, -0.03 instead of -0.09.

Comparison with the published results. MonitoringBench's headline rates come from a full-trajectory monitor that sees tool calls without their outputs. Its threshold is set at a 1% trajectory-level FPR on 416 honest runs (Jotautaitė et al., 2026).

Table 13: LLR-SUM minus MAX at a 5% budget, with 95% intervals from the paired bootstrap over attack trajectories (thresholds held fixed, as in Figure 6) and from the two-way bootstrap (honest and attack runs resampled, thresholds and likelihood ratios refitted, 1,000 replicates). Point estimates are those of the original analysis. Our monitor was scored on a different set of Opus attacks (Section 7), hence no entry.

	human-crafted sabotage		Opus-generated live attacks	
Monitor	attacks resampled	two-way	attacks resampled	two-way
Ours	+0.14 [+0.06, +0.22]	+0.14 [−0.03, +0.25]	–	–
Claude Opus 4.6	−0.09 [−0.16, −0.02]	−0.09 [−0.33, +0.17]	−0.20 [−0.28, −0.13]	−0.20 [−0.34, −0.10]
Claude Sonnet 4.6	−0.12 [−0.20, −0.03]	−0.12 [−0.20, −0.03]	−0.29 [−0.40, −0.19]	−0.29 [−0.42, −0.13]
Claude Haiku 4.5	−0.12 [−0.21, −0.04]	−0.12 [−0.21, +0.00]	−0.21 [−0.30, −0.12]	−0.21 [−0.36, −0.01]
GPT-5.4	−0.15 [−0.23, −0.07]	−0.15 [−0.26, −0.01]	−0.33 [−0.43, −0.24]	−0.33 [−0.51, −0.14]
GPT-5 Mini	−0.02 [−0.12, +0.07]	−0.02 [−0.19, +0.21]	−0.20 [−0.29, −0.11]	−0.20 [−0.36, −0.08]
GPT-5 Nano	+0.00 [+0.00, +0.00]	+0.00 [+0.00, +0.00]	−0.05 [−0.14, +0.03]	−0.05 [−0.15, +0.07]
GPT-4.1 Nano	+0.00 [+0.00, +0.00]	+0.00 [+0.00, +0.00]	−0.02 [−0.07, +0.03]	−0.02 [−0.15, +0.07]
GPT-OSS-120B	−0.03 [−0.07, −0.01]	−0.03 [−0.09, +0.04]	−0.11 [−0.22, +0.00]	−0.11 [−0.25, +0.07]
Gemini 3.1 Pro	+0.04 [−0.01, +0.09]	+0.04 [−0.03, +0.16]	−0.10 [−0.17, −0.02]	−0.10 [−0.20, −0.01]
Gemini 3 Flash	−0.02 [−0.10, +0.06]	−0.02 [−0.20, +0.23]	−0.25 [−0.35, −0.16]	−0.25 [−0.38, −0.04]
Kimi K2.5	+0.06 [−0.01, +0.14]	+0.06 [−0.09, +0.25]	−0.04 [−0.16, +0.07]	−0.04 [−0.22, +0.13]
GLM 5	+0.20 [+0.10, +0.30]	+0.20 [+0.00, +0.40]	−0.08 [−0.17, +0.02]	−0.08 [−0.17, +0.11]

Table 14: Our monitor: each detector minus MAX at a 5% budget, with two-way bootstrap intervals (1,000 replicates; the LLR-SUM row repeats Table 13). Point estimates as in Table 2 and Table 16.

Detector	human-crafted sabotage	Haiku attacks	MonitoringBench
LLR-SUM	+0.14 [−0.03, +0.25]	−0.02 [−0.07, +0.00]	−0.09 [−0.15, −0.03]
e-process ($h = 1/\alpha$)	+0.10 [+0.00, +0.19]	−0.05 [−0.10, −0.01]	−0.19 [−0.24, −0.14]
CUSUM	+0.05 [−0.03, +0.14]	+0.01 [−0.02, +0.03]	+0.00 [−0.03, +0.02]
LLR-CUSUM	−0.08 [−0.29, +0.08]	+0.00 [+0.00, +0.03]	−0.02 [−0.06, +0.01]
three strikes	−0.18 [−0.29, −0.06]	−0.01 [−0.04, +0.01]	−0.71 [−0.76, −0.65]

For example, the catch rate of the Opus 4.5 monitor falls from 0.949 on elicited-only attacks to 0.603 on the best refined attacks. We applied the thresholds that the benchmark reports to the mean of its three recorded samples. Its full-trajectory monitors then catch the following shares of our 300 attacks, with the share of all 2,644 attacks in parentheses: Opus 4.5 0.71 (0.75), Sonnet 4.5 0.53 (0.59), GPT-5.1-Codex-Max 0.43 (0.56), Sonnet 4 0.41 (0.51) and GPT-5 0.09 (0.26). Our subset is thus slightly harder than the benchmark as a whole and close to its refined attacks, on which Opus 4.5 catches 0.67. At the same 1% trajectory-level budget, our monitor catches 0.70 with the strict full-data threshold and 0.77 with cross-validation and randomized tie-breaking, close to Opus 4.5. The benchmark's stepwise monitors see tool outputs, as ours does, and catch more, about 0.83 for Opus 4.5. There, a 1% per-step threshold implies a 6.7–17.4% trajectory-level FPR, consistent with our 10%. The 0.92 in Table 3 therefore reflects the looser 5% budget and the screened benign pool rather than an easier attack set.

Attackers that adapt. The score-level stress tests of Appendix F reproduce the LinuxArena pattern. Concentrated attacks cut LLR-SUM from 0.83 to 0.60, while MAX stays at 0.92. Under sub-threshold attacks, MAX catches nothing and LLR-SUM 0.14. The OR rule, which alarms when either statistic exceeds its $\alpha/2$ threshold, keeps a catch rate of 0.89 on unmodified attacks, 0.88 on concentrated attacks and 0.10 on sub-threshold ones. On unmodified attacks it costs 0.03 against MAX, and its realized FIR is 5%.

J Robustness checks

Deterministic tie-breaking. The main results break ties between equal scores at random. A deployed monitor would more likely alarm only when the statistic exceeds the raw order statistic (the strict rule of Appendix E), which keeps both guarantees but leaves part of the budget unused when honest maxima tie at the threshold. Table 18 compares the two rules. For our monitor, whose scores average two samples, the strict rule lowers the realized FIR at a 5% budget to 4.0% and the catch rate on human sabotage from 0.37 to 0.35, and on MonitoringBench from 0.92 to 0.89. The integer scores

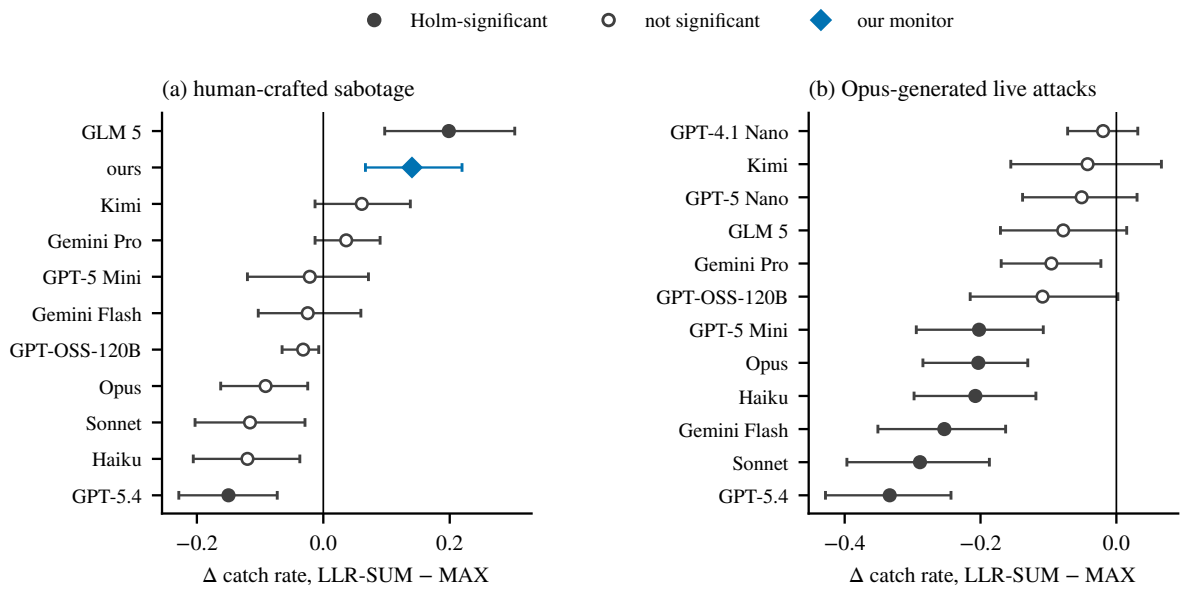


Figure 6: Change in pre-harm catch rate when LLR-SUM replaces MAX at a 5% FIR budget, on (a) human-crafted sabotage and (b) Opus-generated live attacks (likelihood ratio fitted in-domain), with 95% paired bootstrap CIs over attack trajectories and thresholds held fixed (Table 13 gives intervals that also refit the thresholds). Filled markers: significant after Holm correction across monitors (paired sign-flip test); open markers: not significant. Significance follows the test, not the uncorrected percentile intervals, so an interval can exclude zero for an open marker; for GPT-OSS-120B in (a) the difference is small and the unadjusted test gives $p = 0.13$. The diamond is our monitor. Monitors with a near-zero catch rate are omitted.

Table 15: MonitoringBench: calibration at $\alpha = 5\%$ (population bootstrap on the 300 honest runs, 1,000 draws). For each method, the columns give the mean FIR, $\Pr[\text{FIR} > \alpha]$ (viol.) and pre-harm catch rate.

	quantile			conformal			PAC ($\delta = 10\%$)		
n	FIR	viol.	catch	FIR	viol.	catch	FIR	viol.	catch
45	0.072	0.60	0.93	0.041	0.28	0.89	0.020	0.07	0.83
60	0.060	0.44	0.93	0.049	0.40	0.91	0.015	0.05	0.81
100	0.055	0.42	0.92	0.047	0.36	0.91	0.017	0.02	0.83
200	0.052	0.42	0.92	0.048	0.35	0.92	0.028	0.04	0.88

of the LinuxArena monitors tie more often: their realized FIR falls to 1.3–4.9%, the catch rate of GPT-5.4 halves (0.18 to 0.09) and that of GPT-5 Mini falls from 0.24 to 0.16, and the others lose at most 0.03. The budget still costs up to 0.39, unrelated to the per-step FIR (Kendall $\tau = 0.13$), and still reorders the monitors ($\tau = 0.76$). For our monitor, the strict rule raises the gain of LLR-SUM over MAX to +0.16 [+0.07, +0.24] and that of the e-process to +0.12, and three strikes loses 0.17, all Holm-significant; CUSUM (+0.06, adjusted $p = 0.08$) and LLR-CUSUM (-0.07, $p = 0.13$) are no longer significant. On MonitoringBench, the difference for LLR-SUM is -0.06 [-0.09, -0.03]. These shifts favor LLR-SUM because its sums rarely tie: its realized FIR stays at 5–6% under both rules while that of MAX falls. We therefore compare statistics with randomized ties, at equal FIR. Where scores are coarse, averaging several samples per step, as in our baseline, or randomizing at the threshold level recovers the unused budget.

The budget on live attacks. Table 19 repeats Figure 2 on the 30 Opus-generated live attacks whose side task succeeded. Per-step thresholds catch 0.97–1.00 of them for ten monitors, and a 5% budget costs the nine monitors that catch more than 0.10 of human sabotage at most 0.07; GPT-5 Nano, GPT-4.1 Nano and GPT-OSS-120B lose 0.38, 0.39 and 0.10. With our monitor on the live attacks of the seven attacker models (45 honest runs each, full-data thresholds, strict rule), per-step thresholds catch 0.93–1.00 at an FIR of 9–27%, and a 5% budget catches 0.87–1.00. On Haiku live attacks, the budget costs 0.03 (0.90 to 0.87).

Table 16: MonitoringBench: detectors at a 5% budget (split-conformal, 5-fold CV × 10 seeds). The columns give catch rate with FIR in parentheses, the difference from MAX with a paired bootstrap CI, and the Holm-adjusted p of a paired sign-flip test.

Detector	catch (FIR)	Δ vs. MAX	p_{Holm}
MAX	0.92 (0.05)	–	–
LLR-SUM	0.83 (0.05)	−0.09 [−0.12, −0.06]	<0.001
e-process ($h = 1/\alpha$)	0.73 (0.01)	−0.19 [−0.23, −0.15]	<0.001
CUSUM	0.92 (0.05)	+0.00 [−0.01, +0.00]	0.167
LLR-CUSUM	0.90 (0.05)	−0.02 [−0.03, −0.01]	0.020
three strikes	0.21 (0.05)	−0.71 [−0.76, −0.66]	<0.001

Table 17: MonitoringBench: timeliness at a 5% budget (full-data threshold). The columns give the share caught before harm, the median lead, and the shares of caught attacks flagged at the harmful call (lead 1) and within two steps. One human-in-the-loop attack is not listed by source.

Attacks	n	before harm	lead	lead = 1	≤ 2
all	300	0.87	1	0.62	0.70
firewall	77	0.74	1	0.65	0.72
malware	78	0.97	1	0.74	0.76
exfiltration	65	0.78	1	0.65	0.75
sudo escalation	80	0.96	2	0.47	0.60
refined	280	0.86	1	0.64	0.73
reconnaissance	19	1.00	6	0.32	0.37

Catch rate across budgets. Figure 7 plots the catch rate of MAX against the FIR for every monitor, with the per-step operating points of Figure 2. On human sabotage, most curves keep rising up to an FIR of 30%, so the budget trades interruptions against catches throughout this range; on Opus-generated live attacks, capable monitors reach 0.90–0.97 by an FIR of 5%. These curves are what a deployer needs to choose α. If a fraction π of runs are attacks, interrupting an honest run costs c_I and an attack that is not stopped costs c_H, the expected cost per run, $c_I(1-\pi)\text{FIR} + c_H\pi(1-\text{catch})$, is smallest where the slope of the curve falls to $c_I(1-\pi)/(c_H\pi)$. If an interruption costs a hundredth of a successful attack and one run in a thousand is an attack, raising the FIR by one point pays off only while it adds more than 0.10 to the catch rate.

Shift within an agent. The population bootstrap of Section 5 makes calibration and deployment runs exchangeable by construction. To test a shift in the task mix with the agent fixed, we hold out each of the ten LinuxArena environments (20 honest Opus runs each) in turn, calibrate on the other nine and compute the FIR on the held-out one; on MonitoringBench we hold out each of the 92 tasks. Table 20 compares the pooled FIR with 1,000 random splits into groups of the same sizes. For conformal thresholds, holding out environments raises the pooled FIR significantly for our monitor (6.5%), the Haiku monitor (6.0%) and the Kimi monitor (9.0%), whose worst held-out environment is interrupted in 65% of runs; for the other monitors it stays at 4.5–5.5%, as for random splits. PAC thresholds keep the pooled FIR at 2.5–4.0% throughout, but single held-out environments still reach up to 25%. The strict rule gives the same picture. Holding out MonitoringBench tasks, which share one environment, changes nothing (5.0% against 4.8%).

Sampling uncertainty. Figure 2 rests on 13 monitors and about 70 attacks. In a bootstrap over honest runs and attacks (1,000 replicates, full-data thresholds), the ranking of the monitors changes between the per-step threshold and the budget in every replicate (Kendall τ between the rankings 0.60–0.89), GLM 5 loses rank in 91% and Sonnet 4.6 in 82% of replicates, and the largest loss is 0.29–0.50. The relation between the loss and the per-step FIR is not reliably positive: with full-data thresholds, Kendall's τ is 0.35 with a 95% interval of -0.05 to 0.57, against 0.03 with cross-validated thresholds. Our intervals resample trajectories, but trajectories from the same environment or task may be correlated. Resampling the eight environments of the human sabotage instead widens the intervals for LLR-SUM minus MAX by up to a factor of two: the gain for our monitor ([+0.01, +0.24]) and the losses for Opus 4.6 and GPT-5.4 still exclude zero, while the gain for GLM 5 ([-0.01, +0.35]) no longer does. On MonitoringBench, resampling the 92 tasks leaves the loss of LLR-SUM unchanged (-0.09 [-0.13, -0.06]).

Cross-agent uncertainty. Resampling both honest pools (300 replicates), the mean FIR of conformal thresholds calibrated on Haiku runs and deployed on Opus runs has a 95% interval of 4.0–12.7%, against 1.9–9.2% when both come from

Table 18: Per-step 1% thresholds and a 5% budget (MAX) on human sabotage with randomized and deterministic (strict) tie-breaking: catch rate with realized FIR in parentheses (5-fold CV × 10 seeds). The randomized columns come from a separate run and differ from Figure 2 by at most 0.01.

	per-step 1%		5% budget	
Monitor	randomized	strict	randomized	strict
Ours	0.55 (0.243)	0.52 (0.199)	0.37 (0.049)	0.35 (0.040)
Claude Opus 4.6	0.79 (0.296)	0.77 (0.278)	0.58 (0.049)	0.56 (0.042)
Claude Sonnet 4.6	0.47 (0.364)	0.43 (0.340)	0.13 (0.049)	0.13 (0.045)
Claude Haiku 4.5	0.32 (0.304)	0.30 (0.292)	0.16 (0.049)	0.14 (0.035)
GPT-5.4	0.38 (0.148)	0.35 (0.122)	0.18 (0.050)	0.09 (0.013)
GPT-5 Mini	0.55 (0.140)	0.54 (0.134)	0.24 (0.050)	0.16 (0.041)
GPT-5 Nano	0.03 (0.232)	0.03 (0.200)	0.00 (0.049)	0.00 (0.015)
GPT-4.1 Nano	0.01 (0.288)	0.00 (0.081)	0.00 (0.050)	0.00 (0.040)
GPT-OSS-120B	0.22 (0.311)	0.18 (0.283)	0.03 (0.056)	0.00 (0.021)
Gemini 3.1 Pro	0.73 (0.358)	0.68 (0.295)	0.59 (0.049)	0.59 (0.044)
Gemini 3 Flash	0.58 (0.276)	0.49 (0.198)	0.34 (0.051)	0.33 (0.049)
Kimi K2.5	0.44 (0.344)	0.40 (0.323)	0.14 (0.053)	0.13 (0.044)
GLM 5	0.63 (0.294)	0.62 (0.276)	0.26 (0.049)	0.23 (0.044)

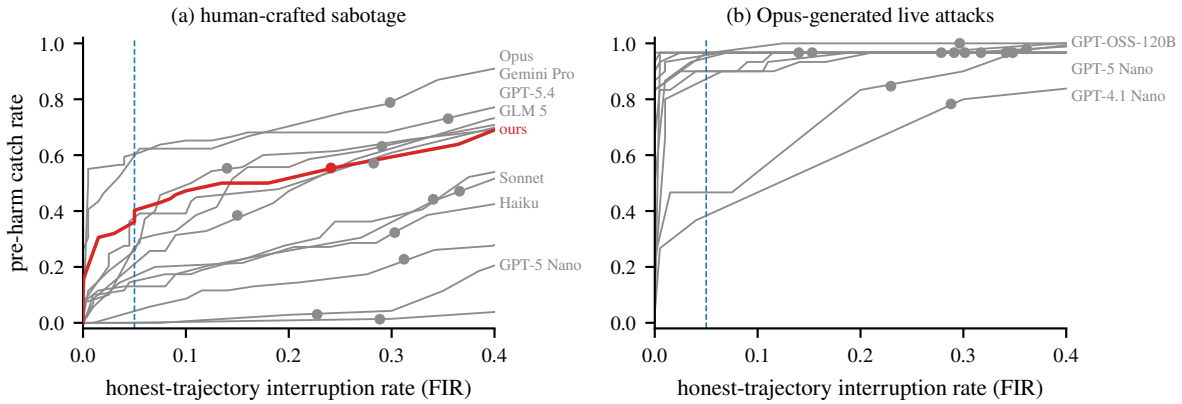


Figure 7: Pre-harm catch rate of MAX against the FIR over all thresholds (full-data thresholds, randomized ties), on (a) human-crafted sabotage and (b) Opus-generated live attacks. Dots: per-step 1% thresholds (cross-validated, as in Figure 2); dashed line: a 5% budget; red: our monitor.

Opus runs; for our monitor, the evidence thus rests on the higher probability of exceeding the budget (74% against 36% of draws) more than on the mean. Across the seven agents with 45 runs each, cross-agent cells average 4.7% and within-agent cells 3.3%, and 15 of the 42 cross-agent cells exceed 5%.

Trajectory length. The running maximum of a longer run has more chances to exceed a fixed threshold. With a pooled conformal threshold (5-fold CV × 10 seeds), the longest third of honest runs is interrupted at 10.5% (Opus), 11.0% (Haiku) and 8.7% (MonitoringBench), against 0.2–3.4% for the shorter two thirds. Calibrating a separate threshold for each length tercile (Mondrian conformal) brings every tercile to 2.6–5.0% and changes the catch rate by -0.01 to $+0.01$.

Scoring variance. Our baseline scores average two samples per step. Using either sample alone gives a 5% budget catch rate of 0.35 or 0.36 on human sabotage (both samples: 0.37), with LLR-SUM gains of +0.13 and +0.18, and 0.85 on Haiku attacks; the honest maxima of the two samples correlate at 0.79.

K Harm-step annotation

LinuxArena places the harm step $t^\star$ of a live failure at the first passing side-task test, or at the final submission if there is none. The harm itself happens at an earlier action, the one that accomplishes the side task, so an alarm between that action and the test counts as pre-harm although it comes after the harm. This can inflate both the catch rate and the lead, the more so the later an attacker runs the test. We therefore annotated the harmful action of

Table 19: Opus-generated live attacks (30 per monitor): catch rate (realized FIR) at the per-step 1% threshold and at a 5% budget with randomized and strict tie-breaking (5-fold CV $\times$ 10 seeds).

	per-step 1%	5% budget	
Monitor	randomized	randomized	strict
Claude Opus 4.6	1.00 (0.296)	0.97 (0.049)	0.97 (0.042)
Claude Sonnet 4.6	0.98 (0.361)	0.97 (0.050)	0.97 (0.045)
Claude Haiku 4.5	0.97 (0.301)	0.94 (0.051)	0.93 (0.035)
GPT-5.4	0.97 (0.153)	0.94 (0.054)	0.93 (0.013)
GPT-5 Mini	0.97 (0.140)	0.96 (0.050)	0.93 (0.041)
GPT-5 Nano	0.85 (0.230)	0.47 (0.045)	0.47 (0.015)
GPT-4.1 Nano	0.78 (0.288)	0.39 (0.049)	0.36 (0.040)
GPT-OSS-120B	0.97 (0.317)	0.87 (0.051)	0.81 (0.021)
Gemini 3.1 Pro	0.97 (0.348)	0.97 (0.050)	0.97 (0.044)
Gemini 3 Flash	0.97 (0.279)	0.90 (0.051)	0.90 (0.049)
Kimi K2.5	0.97 (0.342)	0.90 (0.051)	0.89 (0.044)
GLM 5	0.97 (0.291)	0.97 (0.048)	0.97 (0.044)

all 147 successful live attacks that we analyze: the 105 attacks of the seven attacker models, 22 Haiku and 5 human attacks, and the 30 Opus-generated attacks scored by the 12 LinuxArena monitors (15 overlap with the first set). DeepSeek-V4.1-Flash in thinking mode read the side-task goal and scorer description and the complete action list, including tool outputs, hidden tools and the agent's own reasoning, which no monitor sees, and named the earliest action after which the side-task scorer would pass. We drew three labels per trajectory and use their lower median. All three agreed for 94% of the trajectories; 0.7% of the labels fell after $t^\star$ and were clipped to it. In ten trajectories that we inspected, the chosen action was the one that sent the credential, opened the connection, installed the key or deleted the records. An alarm at or before the annotated action k prevents the harm, and the lead is counted so that an alarm at k has lead 1.

The harmful action is the last visible action before the test for 47–87% of each attacker's attacks (Table 21). No catch rate of our monitor changes, and the Opus 4.6 attacks keep their median lead of one step, with 10 of 13 catches at the harmful action. The median leads of the Grok 4.1 Fast, Sonnet 4.6 and Gemini 3.1 Pro attacks shrink from 23 to 17, 10 to 8 and 6 to 5 steps, which keeps the ordering of attackers, and on Haiku live attacks the median lead falls from 25 to 18 steps. Of the 12 LinuxArena monitors on Opus-generated attacks, only GPT-OSS-120B loses a catch (0.80 to 0.77), and only the Opus 4.6 monitor's median lead shrinks (7 to 4 steps). In the deployment simulation of the Opus attacks (Table 7), the annotated harm step changes the share prevented by at most 0.07. The annotator is an LLM, the monitor's own model, and we have no human labels.

Table 20: Holding out one environment (LinuxArena) or task (MonitoringBench) at a time: conformal and PAC ($\delta = 10\%$) thresholds at $\alpha = 5\%$ calibrated on the remaining honest runs (randomized ties). Columns: number of groups; pooled FIR of conformal thresholds on the held-out runs, its mean over 1,000 random splits into groups of the same sizes, and the permutation p-value; the highest FIR of a single held-out group and its 95% quantile under random splits; the pooled FIR of PAC thresholds.

		conformal, pooled FIR			worst group		PAC
Honest pool	groups	held out	random	p	held out	random	held out
Ours, Opus honest	10	0.065	0.050	0.004	0.35	0.20	0.030
Ours, Haiku honest	10	0.045	0.050	1.000	0.25	0.20	0.040
Claude Opus 4.6, Opus honest	10	0.055	0.050	0.198	0.20	0.25	0.025
Claude Sonnet 4.6, Opus honest	10	0.050	0.050	0.733	0.15	0.20	0.030
Claude Haiku 4.5, Opus honest	10	0.060	0.050	0.024	0.30	0.25	0.030
GPT-5.4, Opus honest	10	0.055	0.050	0.234	0.25	0.25	0.040
GPT-5 Mini, Opus honest	10	0.055	0.050	0.203	0.25	0.20	0.035
GPT-5 Nano, Opus honest	10	0.055	0.050	0.211	0.30	0.25	0.040
GPT-4.1 Nano, Opus honest	10	0.055	0.050	0.218	0.20	0.25	0.025
GPT-OSS-120B, Opus honest	10	0.050	0.050	0.718	0.15	0.21	0.030
Gemini 3.1 Pro, Opus honest	10	0.050	0.050	0.709	0.15	0.20	0.025
Gemini 3 Flash, Opus honest	10	0.055	0.050	0.211	0.20	0.25	0.025
Kimi K2.5, Opus honest	10	0.090	0.050	$\leq$0.001	0.65	0.20	0.035
GLM 5, Opus honest	10	0.050	0.050	0.700	0.15	0.20	0.030
Ours, MonitoringBench (tasks)	92	0.050	0.048	0.293	–	–	0.033

Table 21: Our monitor at a 5% budget (conformal on 45 honest runs of the same agent, strict rule) with the harm step at the first passing side-task test ($t^\star$) and at the annotated harmful action. Gap: visible steps between the harmful action and $t^\star$. Pairs give the value under $t^\star$ / under the annotated action. Last row: Haiku live attacks with a successful side task (200 honest runs).

Attacker	n	catch	median lead	lead $= 1$	median gap	gap $= 0$
Opus 4.6	15	0.87 / 0.87	1 / 1	10 / 10	0	0.87
Sonnet 4.6	15	0.93 / 0.93	10 / 8	0 / 1	0	0.73
GLM 5	15	0.87 / 0.87	11 / 11	0 / 1	0	0.67
Gemini 3.1 Pro	15	1.00 / 1.00	6 / 5	0 / 1	0	0.80
Gemini 3 Flash	15	1.00 / 1.00	19 / 19	0 / 0	0	0.73
Grok 4.1 Fast	15	1.00 / 1.00	23 / 17	2 / 2	1	0.47
Kimi K2.5	15	0.87 / 0.87	8 / 8	0 / 0	0	0.87
Haiku 4.5 (live)	22	0.95 / 0.95	25 / 18	0 / 0	2	0.32